

**An Integrated Machine Learning Framework for Crop
Selection, Fertilizer Suggestion, and Disease Diagnosis.**

*A
Project Report*

*Submitted in partial fulfilment of the
Requirements for the award of the Degree of*

BACHELOR OF ENGINEERING

IN

INFORMATION TECHNOLOGY

By

1602-21-737-058 Suchith.B

1602-21-737-015 Deepthi.M

Under the guidance of

C.Sireesha

Assistant Professor



**Department of Information Technology
Vasavi College of Engineering (Autonomous)
ACCREDITED BY NAAC WITH 'A++' GRADE
(Affiliated to Osmania University)
Ibrahimbagh, Hyderabad-31
2025**

Vasavi College of Engineering (Autonomous)

ACCREDITED BY NAAC WITH 'A++' GRADE

(Affiliated to Osmania University)

Hyderabad-500 031

Department of Information Technology



DECLARATION BY THE CANDIDATE

We, **Suchith Bollam** and **Deepthi Modugula** bearing hall ticket numbers, **1602-21-737-058** and **1602-21-737-015** hereby declare that the project report entitled **An Integrated Machine Learning Framework for Crop Selection, Fertilizer Suggestion, and Disease Diagnosis**, under the guidance of **C.Sireesha, Assistant Professor**, Department of Information Technology, Vasavi College of Engineering, Hyderabad, is submitted in partial fulfilment of the requirement for the award of the degree of **Bachelor of Engineering in Information Technology**.

This is a record of bonafide work carried out by us and the results embodied in this project report have not been submitted to any other university or institute for the award of any other degree or diploma.

Suchith Bollam
1602-21-737-058
Deepthi Modugula
1602-21-737-015

Vasavi College of Engineering (Autonomous)

ACCREDITED BY NAAC WITH 'A++' GRADE

(Affiliated to Osmania University)

Hyderabad-500 031

Department of Information Technology



BONAFIDE CERTIFICATE

This is to certify that the project entitled **An Integrated Machine Learning Framework for Crop Selection, Fertilizer Suggestion, and Disease Diagnosis** being submitted by **Suchith Bollam** and **Deepthi Modugula** bearing hallticket numbers **1602-21-737-058** and **1602-21-737-015** in partial fulfilment of the requirements for the award of the degree of Bachelor of Engineering in Information Technology is a record of bonafide work carried out by them under my guidance.

C.Sireesha
Assistant Professor
Internal Guide

External Examiner

Dr. K. Ram Mohan Rao
Professor & HOD, IT

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of the Main project would not have been possible without the kind support and help of many individuals. We would like to extend our sincere thanks to all of them.

It is with immense pleasure that we would like to take the opportunity to express our humble gratitude to **C.Sireesha , Assistant Professor, Department of Information Technology**, under whom we executed this project. Her constant guidance and willingness to share their vast knowledge made us understand this project and its manifestations in great depths and helped us to complete the assigned tasks.

We are very much thankful to **Dr.K.Ram Mohan Rao, H.o.D, Department of Information Technology**, for his kind support and for providing necessary facilities to carry out the work.

We wish to convey our special thanks to **S.V.Ramana , Principal of Vasavi College of Engineering and Management** for providing facilities. Not to forget, we thank all other faculty and non-teaching staff, and my friends who had directly or indirectly helped and supported me in completing my project in time.

ABSTRACT

Agriculture is the backbone of India's economy, providing both income and employment to a large portion of the population. However, many farmers struggle with choosing the right crops and fertilizers for their land, which directly impacts productivity. Precision agriculture offers a solution by using data-driven insights on soil quality, crop suitability, and yield patterns to guide farmers in making informed decisions. By tailoring recommendations to specific land conditions, this approach minimizes errors in crop selection and improves overall farm output.

In this study, we present a machine learning-based recommendation system designed to support farmers with accurate crop selection. Our model employs a majority voting technique that combines the strengths of Random Forest and XGBoost to enhance reliability and efficiency. Additionally, we integrate plant disease detection using deep learning models—CNN, MobileNet, and ResNet—helping farmers identify and address crop diseases early. By leveraging advanced AI techniques, our system aims to make precision agriculture more accessible, ultimately improving decision-making and boosting productivity in farming communities.

TABLE OF CONTENTS

LIST OF FIGURES.....	1
LIST OF TABLES.....	1
LIST OF ABBREVIATIONS	1
1 INTRODUCTION.....	1
1.1 Problem Statement	1
1.2 Proposed Work.....	1
1.3 Scope & Objectives of Proposed Work	2
1.4 Organization of the Report	2
2. LITERATURE SURVEY.....	7
3. PROPOSED SYSTEM.....	14
3.1 Methodology	14
3.1.1 Block Diagram.....	15
3.1.2 Functional Modules.....	16
3.2 Algorithms.....	22
3.1.3 Crop Recommendation.....	22
3.1.4 Fertilizer Recommendation.....	23
3.1.5 Disease Detection.....	25
4 EXPERIMENTAL SETUP AND RESULTS	32
Datasets.....	32
System Specifications	33
Results and Test Analysis	35
5 CONCLUSION AND FUTURE SCOPE	46
REFERENCES.....	47
APPENDIX.....	51
Code.....	51
Plagiarism Report Last Page.....	61
Research Paper.....	62

LIST OF FIGURES

Fig. No	Description	Page No
Figure 3.1	Block Diagram of Proposed System.....	15
Figure 3.2	Architecture for Resnet Model	26
Figure 3.3	Architecture for Mobilenet model	29
Figure 3.4	Architecture for CNN	30
Figure 4.1	Input for Crop Recommendation	36
Figure 4.2	Output shows the crop predicted for the given input values.....	37
Figure 4.3	Input for Fertilizer Recommendation.....	38
Figure 4.4	Output for Fertilizer Recommendation.....	39
Figure 4.5	Input for Disease Detection	40
Figure 4.6	Output of Predicted Disease	40
Figure 4.7	Accuracy vs. no.of epochs graph for XGBoost Algorithm	422
Figure 4.8	Accuracy vs no.of epochs graph for XGBoost and Random Forest.....	43
Figure 4.9	Accuary vs No.of Epochs graph for Resnet Model	433
Figure 4.10	Accuracy vs No.of Epochs for MobileNet Model.....	44
Figure 4.11	Accuracy vs No.of Epochs for CNN model	45
Figure 4.12	BarChart for comparision of Accuracy of different models for disease detection.....	45

LIST OF TABLES

Table No.	Table Name	Page No
2.1	Comparison of Research papers Studied	7
4.3.1	Accuracy of models for Crop and Fertilizer Recommendation.	41
4.3.2	Accuracy of different models for Disease Detection.	41

LIST OF ABBREVIATIONS

1. XGBoost - Extreme Gradient Boost
2. CNN - Convolutional Neural Networks
3. ResNet - Residual Neural Network

1 INTRODUCTION

1.1 Problem Statement

Agriculture is a crucial sector in developing nations, providing livelihoods and ensuring food security. However, farmers face significant challenges such as unpredictable weather, soil degradation, pest infestations, and crop diseases, all of which can severely impact yield and profitability. To tackle these issues, farmers need access to reliable, data-driven insights that aid in making informed decisions about crop selection, soil management, and disease prevention.

This study introduces an Agricultural Crop Recommendation and Disease Detection System powered by machine learning. The crop recommendation module leverages XGBoost and Random Forest algorithms to analyze key agricultural factors such as soil composition, climate conditions, and historical crop yields, providing farmers with personalized recommendations on the most suitable crops for their land. Additionally, a plant disease detection module has been integrated using deep learning models—CNN, MobileNet, and ResNet—to identify and classify crop diseases accurately. By combining machine learning and deep learning, this system enhances decision-making in farming, helping to improve productivity, reduce losses, and promote sustainable agricultural practices.

1.2 Proposed Work

The agricultural crop recommendation system aims to support farmers in selecting the most suitable crops based on critical factors such as soil quality, climate conditions, and geographical location. By leveraging both content-based and collaborative filtering techniques, the system analyzes extensive agricultural data to assess crop performance in specific regions.

This system provides farmers with personalized crop recommendations, considering key parameters like nitrogen (N), phosphorus (P), potassium (K) levels, soil type, rainfall, and temperature. Additionally, it suggests optimal fertilizers to

enhance crop yield and soil fertility. To ensure high accuracy and efficiency, the recommendation engine integrates machine learning algorithms such as XGBoost and Random Forest for data-driven decision-making. Furthermore, a plant disease detection module powered by CNN, MobileNet, and ResNet is incorporated to identify and diagnose crop diseases, enabling timely intervention and reducing potential losses.

1.3 Scope & Objectives of Proposed Work

[1] To build a robust model to provide correct and accurate prediction of crop sustainability in a given state for the particular soil type and climatic conditions.

[2] Provide recommendation of the most effective suitable crops within the area in order that the farmer doesn't incur any losses.

[3] Provide fertilizer suggestion for crops supported chemical features.

[4] To simplify the task of identifying diseases for farmers by using technology that is more effective than the naked eye.

1.4 Organization of the Report

This report has been thoughtfully structured into a series of chapters, each one playing a specific role in building a clear and complete picture of our project: “*An Integrated Machine Learning Framework for Crop Selection, Fertilizer Suggestion, and Disease Diagnosis.*” The organization is meant to guide the reader step-by-step — starting from the problem background and ending with the technical results and future possibilities. Here's how the report is laid out:

Introduction

The report begins by laying the foundation for the project. It explains why agriculture is not just a sector but a backbone of our economy — especially in a country like India. We explore the real struggles that farmers face in the field: how to choose the right crop for their land, how to identify which nutrients the soil lacks, and how to spot

diseases in plants before it's too late. These challenges are deeply connected to productivity and sustainability. The introduction also shares our motivation for tackling these problems through technology — especially machine learning — and clearly states the goals and scope of our work. It gives the reader a good understanding of what to expect in the rest of the report.

Literature Survey

In this chapter, we explore what's already been done in the field. We look at previous research studies and projects that tried to solve similar problems — from crop prediction using SVM and Random Forest, to disease detection using deep learning models like CNNs and ResNet. By comparing various approaches, models, and their results, we highlight the strengths and limitations of each. This helps us justify our choice of methods and shows that our work builds on a strong foundation. We also identify gaps in earlier systems — such as lack of integration, limited datasets, or low accuracy — and use that as motivation to design a better solution.

Problem Definition

This section clearly defines what problem our project aims to solve. Instead of addressing one isolated issue, we focus on three interconnected challenges that farmers often face together — deciding what to grow, what nutrients the soil needs, and whether the crops are healthy. We explain why it's important to build an integrated system that can handle all three tasks, so that farmers don't need to rely on separate tools. This section breaks the problem down into simple, real-world terms, helping readers (even non-technical ones) connect with the project's purpose.

Proposed System

This is the heart of the report where we present our solution. We describe the system's overall architecture and how it has been divided into three modules: crop recommendation, fertilizer suggestion, and plant disease detection. Each module is discussed in detail — explaining how user inputs (like soil data or leaf images) are processed using machine learning and deep learning models. We explain why we selected specific algorithms like XGBoost for crop prediction, and why ResNet gave better results for disease detection compared to CNN or MobileNet. We also include

block diagrams and visuals to help the reader understand the flow of the system from input to output.

System Architecture

This section bridges the technical and user-friendly aspects of the system. We explain how the system has been designed from a software architecture perspective. It includes the flow of data between the front-end (what the farmer sees), the back-end (where predictions are made), and the machine learning models (where decisions happen). We talk about the technologies used, such as Python for development, Flask for the web application, and HTML/CSS for the interface. The goal of this section is to show how all components — algorithms, datasets, UI, and logic — come together in a seamless and scalable application.

Datasets and Preprocessing

Here, we introduce the datasets we used — all of which were sourced from trusted platforms like Kaggle. We explain what kind of data was used for each module: soil parameters for crop recommendation, nutrient levels for fertilizer suggestions, and leaf images for disease classification. The preprocessing steps, like cleaning data, normalizing values, and preparing images (resizing, augmentation, etc.), are described in detail. This section helps the reader understand the quality of our data and how we prepared it to ensure good model performance.

Experimental Setup and Results

In this chapter, we shift focus from design to results. We describe the system configuration — including the software and hardware requirements used during model training and testing. Then we walk through the performance results of each model — showing tables, graphs, and interface screenshots. For example, the crop recommendation module achieved over 99% accuracy with XGBoost, while ResNet gave over 99% accuracy in disease detection. We include Accuracy vs Epoch graphs and comparison charts to make the performance easy to visualize. These results confirm that our solution is not just theoretically sound but also practically effective.

Performance Evaluation

This section goes a level deeper into analyzing the results. Instead of just reporting accuracy, we compare models across different criteria: speed, reliability, scalability, and usability. We explain why certain models were chosen over others — for example, why ResNet was better suited than MobileNet for image classification despite being slightly more resource-intensive. The aim here is to critically assess our own system and show that model selection wasn't random, but based on careful testing and evaluation.

Conclusion and Future Scope

This final chapter brings the report to a close. We summarize what we achieved — developing an AI-based framework that helps farmers make better decisions about crops, fertilizers, and diseases. We reflect on how our system performs well and can make a real-world impact. Then we look ahead into the future, discussing possibilities like integrating IoT for live soil monitoring, building a mobile app to reach more farmers, or expanding the model to support multiple languages. The goal is to show that this project is not the end but the beginning of something with long-term potential.

References

The "Sources" section highlights the various resources that played a crucial role in shaping and guiding our project. Throughout the process, we drew insights from a wide range of academic research papers, journal articles, and conference proceedings, which provided valuable knowledge on machine learning techniques, precision agriculture, and plant disease detection. These resources offered a solid theoretical foundation and were essential to understanding the complexities of our work.

Furthermore, publicly available datasets, particularly those from platforms like Kaggle, played a pivotal role in training and evaluating our models. These datasets not only helped improve the accuracy of our work but also allowed us to test our solutions under real-world conditions.

Citing these sources is important, as it acknowledges the hard work of the original authors and provides credibility to our own work. This section reflects the foundational resources that helped bring our project to life and offers valuable pointers for others who may continue in this field of research.

Appendices

Finally, we include additional content that supports our main chapters — such as pseudo-code for key algorithms, the plagiarism report, and our research paper submission. These appendices are useful for anyone who wants to dig deeper into the technical details of our project.

2. LITERATURE SURVEY

Rakesh, D. C., Vardhan, V., Vasantha, B. B., & Krishna et al.[1] (2023). Crop recommendation and prediction system. *Proceedings of the 9th International Conference on Advanced Computing and Communication Systems (ICACCS)*, 2023. the authors developed a system to recommend and predict suitable crops based on various parameters. They employed machine learning algorithms, including Support Vector Machine (SVM) and Decision Tree, to create models that provide informed and precise crop selection recommendations based on soil health and food price data.

Takalani Orifha Mufamadi et al.[2] In the study "Crop Recommendation Using Machine Learning Algorithms and Soil Attributes Data," co-authored with Ritesh Ajoodha, various machine learning models were evaluated for crop recommendation. The study focused on soil attributes such as nitrogen, potassium, phosphorus, and pH levels. It was found that individual models like Random Forest and Gaussian Naïve Bayes performed exceptionally well, achieving accuracies close to 99%, while hybrid models, particularly the SVM and Gaussian Naïve Bayes combination, also demonstrated strong predictive performance. Crop Recommendation using Machine Learning Algorithms and Soil Attributes, various machine learning models were evaluated for crop recommendation. The study focused on soil attributes such as nitrogen, potassium, phosphorus, and pH levels.

Table 2.1 Comparison of Research papers Studied

S.No	Author	Approach	Highest Accuracy	Limitaions
1	CH. Rakesh, D. Vishnu Vardhan, Babu Bhavani Vasantha, G. Sai Krishna	KNN, Decision Tree,Random Forest,SVM	97.28%	Overfitting Risk

2	Takalani Orifha Mufamadi	Knn, Random Forest, K-Star, Naive Bayes	91.30%	Validation methods
3	Priyadharshini A	KNN ,Logistic Regression,SVM ,Naive Bayes	95.00%	Feature selection and preprocessing
4	Ashima Kukkar	Naive Bayes, Random Forest, SVM, KNN, Decision Trees	97.07%	Computational requirements
5	Rakesh Kumar	KNN, Naïve Bayes ,Random Forest, K-Star	96.00%	Computational requirements
6	A Mythili	KNN, Logistic Regression ,SVM ,Bagged Tree	91.09%	Feature selection and preprocessing
7	<u>S. Pudumalar; E. Ramanujam; R. Harine Rajashree</u>	SVM, Random Forest, KNN, Bagging Technique And Naïve Bayes	90.75%	Data Imbalance
8	Gayathri,Saranya, Brindha	Random Forest, K-means Clustering	-	No reported accuracy
9	Shwetha A N, Prabodh C P	Random Forest, K Nearest Neighbor and Naive Baye's	95%	Preprocessing
10	Ajay Agarwal, Sartaj Ahmad, Adesh Pandey	DecisionTree, NaiveBayes, SVM,Logistic Regression,RandomFore st	95%	Data Imbalance
11	<u>Zhiyong Xiao</u>	Lightweight model, Resnet for leaf disease detection	95.71%	High computational power
12	Divine Senanu Ametefe	DenseNet201 and EfficientNetB3	96.03% & 92.23%	High computation
13	<u>Aisha Ahmed AlArfaj</u>	UNet segmentation, and the InceptionV3 model	93.48%	Only bell paper
14	Kulkarni	For feature extraction - Gabor filter and for classification- ANN classifier.	91%	Lesser accuracy

15	Emanoul Cortes	Generative Adversarial networks, Background segmentation	80%	Accuracy of background-based segmentation did not rise
16	Hassouni	CNN model with hand crafted features and trained a support vector machine (SVM) classifier	95.65%	Model needs more Enhancement
17	Mohanty	SVM, other machine learning techniques.	86.62%	Model needs more optimization
18	K.Muthukan nan	Radial Basis Function Networks and Feed Forward Neural Networks.	-	This method cannot be applied to all plants.
19	Kaur & Singh	CNNs: AlexNet, VGG16, ResNet50	95.71% (ResNet50)	This method cannot be applied to all leaves
20	Vaishnavi et al.	Custom CNN	90%	Sensitivity to image quality

Priyadharshini A et al.[3]. Crop recommendation using machine learning techniques. International Journal of Engineering Research & Technology (IJERT). The research highlights the potential of machine learning algorithms, such as decision trees, K-Nearest Neighbors (KNN), Random Forests, and neural networks, in analyzing complex agricultural data. Among these, neural networks demonstrated the highest accuracy in predicting suitable crops based on the input parameters.

Kukkar, A., Mohana, R., & Sharma, et al.[4] AgroAdvisor: Crop Yield Prediction, Crop and Fertilizer Recommendation System using Random Forest with Gradient Boosting and for Precise Agriculture. *Research Square*.

Rakesh Kumar et al.[5]Employed KNN, Naïve Bayes, Random Forest, and K-Star in analysis for the crop recommendation according to the soil conditions.

Mythili A et al.[6] Classifying Soil Based on Nutrients and Predicting Suitable Crops Using Machine Learning Techniques. *Journal of Agriculture and Horticulture*. Literature highlights the importance of soil nutrients in crop productivity, with several studies showing the effectiveness of algorithms like KNN, SVM, and logistic regression for soil and crop prediction. This paper extends these approaches by applying multiple machine learning techniques to improve soil classification and optimize crop selection based on nutrient content.

Pudumalar, S., Ramanujam, E., & Rajashree et al.[7] Precision Agriculture for Crop Prediction Using Machine Learning Techniques. *Journal of Agricultural Systems*, with algorithms like SVM and Random Forest being frequently employed due to their ability to handle complex, non-linear relationships in agricultural data. This paper extends the existing body of work by integrating multiple models to improve the accuracy of crop prediction and provide practical solutions for farmers.

Gayathri, S., Saranya, & Brindha et al.[8]. Crop Prediction and Classification Using Random Forest and K-means Cl. *International Journal of Agricultural Systems*: Machine learning algorithms, particularly Random Forest and K-means Clustering, have been widely used in agricultural research. Random Forest has proven effective in handling high-dimensional agricultural data, while K-means Clustering helps in identifying patterns and groupings within clustering large datasets. Previous research has shown that these techniques can significantly improve crop prediction accuracy and resource allocation in agriculture.

Shwetha A N, & Prabodh C et al.[9] Crop Prediction Using Random Forest, K-Nearest Neighbor, and Naïve Bayes. *Journal of Agricultural Data Science*, Random Forest, KNN, and Naïve Bayes have been widely adopted in agricultural research due to their effectiveness in handling large, diverse datasets and their ability to provide interpretable results. Previous studies have demonstrated that these machine learning models can significantly improve crop prediction accuracy, leading to more efficient resource management and better crop yield.

Agarwal, A., Ahmad, S., & Pandey et al.[10] Crop Prediction Using Decision Tree, Naïve Bayes, SVM, Logistic Regression, and Random Forest. *Agricultural Data Science Journal*, Machine learning techniques, including Decision Trees, Naïve Bayes, SVM, Logistic Regression, and Random Forest, have been extensively used in agricultural research to improve crop prediction accuracy. Previous studies have shown that ensemble methods like Random Forest and models such as SVM and Decision Trees can effectively handle the complexity of agricultural data, providing actionable insights for farmers.

Zhiyong Xiao et al. [11] focused on utilizing a lightweight model with ResNet for detecting leaf diseases. The study achieved an impressive accuracy of 96.71% using the Plant Village Dataset. However, a significant limitation noted in the research was the high computational power required, which makes it difficult to implement on resource-constrained devices. This limitation may affect the scalability and practical applicability of the model in real-world environments where computing resources are limited.

Divine Senanu Ametefe et al. [12] employed DenseNet201 and EfficientNetB3 architectures for leaf disease detection. The DenseNet201 model achieved an accuracy of 96.03%, while EfficientNetB3 recorded an accuracy of 92.23%. Both models were trained using the Plant Village Dataset. Despite the high accuracy, the study highlighted the significant computational demands required for these models, making them less feasible for deployment on devices with limited computational resources. This issue could hinder their practical use in applications that need real-time processing.

Aisha Ahmed AlArfaj et al.[13] used UNet segmentation and the InceptionV3 model to detect leaf diseases, with an accuracy of 93.48%. The Bell Pepper Dataset was employed for training the model, which demonstrated promising results in detecting bell pepper diseases. However, the major limitation of the study is that the model is tailored specifically for bell pepper disease detection and does not

generalize well to other types of plants or diseases. This restricts the applicability of the model to a broader range of crops.

Kulkarni et al. [14] introduced a model that used a Gabor filter for feature extraction and an Artificial Neural Network (ANN) classifier for disease classification. The model achieved a moderate accuracy of 91%, but it faced challenges in terms of achieving higher performance. The dataset used was custom, and while the model performed adequately, the accuracy was relatively low compared to other advanced deep learning models. As a result, the model requires further refinement to improve its detection accuracy and generalization.

Emaneul et al. [15] Cortes explored the use of Generative Adversarial Networks (GANs) for background segmentation in leaf disease detection. The model reported an accuracy of 80%, which is relatively lower compared to other methods. The dataset used for training consisted of 86,147 images of healthy and diseased plants from a publicly available collection. Despite the efforts, the accuracy of background-based segmentation did not significantly improve, indicating that the model requires further enhancement to achieve better results in complex scenarios.

Hassouni et al. [16] proposed a model combining Convolutional Neural Networks (CNN) with handcrafted features and a Support Vector Machine (SVM) classifier for disease detection. The model achieved a high accuracy of 95.65% using a dataset of tomatoes. However, the model has some limitations, including the need for further enhancements to improve its generalization and performance across different datasets. This indicates that while the model performs well with tomatoes, it may struggle to generalize to other plant species or diseases.

Mohanty et al. [17] utilized Support Vector Machines (SVM) along with other machine learning techniques for leaf disease detection. The model achieved an accuracy of 86.62%, and it used a custom dataset that includes five different plant diseases based on color and texture features. While the model performed well, it was noted that it requires more optimization to handle a wider range of plant

diseases and improve accuracy. The study emphasized the need for better feature selection and model refinement to achieve more robust results.

K.Muthukan et al. [18] employed Radial Basis Function Networks and Feed Forward Neural Networks for leaf disease classification. Unfortunately, the study did not report an accuracy, but the model utilized custom texture data from images of diseased leaves. The primary limitation of the model is that it cannot be applied to all leaf types, restricting its use to specific plant species and requiring modifications to work with a broader range of crops.

Kaur & Singh et al. [19] utilized various CNN architectures, including AlexNet, VGG16, and ResNet50, for disease detection in plants. The ResNet50 model achieved an accuracy of 95.71%, and the dataset used included images of tomato, potato, and apple leaves. Although the model performed well, it faced limitations due to the relatively small size of the custom dataset, which could lead to overfitting. As a result, the model may not generalize well to unseen data or new plant species, highlighting the importance of larger, more diverse datasets for improving performance.

Vaishnavi et al. [20] developed a custom CNN model for leaf disease detection, achieving an accuracy of 90%. The model used a custom dataset for training. However, the model's performance was sensitive to image quality, including factors such as lighting conditions and resolution. This sensitivity can negatively impact the model's robustness, especially in real-world scenarios where image quality may vary significantly.

3. PROPOSED SYSTEM

3.1 Methodology

Our system is designed to help farmers make smart decisions using a step-by-step process powered by machine learning and deep learning. It starts with a Crop Recommendation System. Farmers provide basic data like soil nutrients (NPK), temperature, pH, and rainfall, usually stored in a CSV file. This data is analyzed using an XGBoost model to recommend the most suitable crops for their land.

Next comes the Fertilizer Recommendation System, which looks at soil condition data to suggest the right type of fertilizer. Here, we use a mix of XGBoost and Random Forest algorithms to make accurate fertilizer suggestions that match the soil's needs.

The system also helps with identifying plant diseases. Farmers can upload images of affected plants, and a ResNet deep learning model analyzes the images to detect any disease. If a disease is found, the system recommends a suitable cure or treatment.

3.1.1 Block Diagram

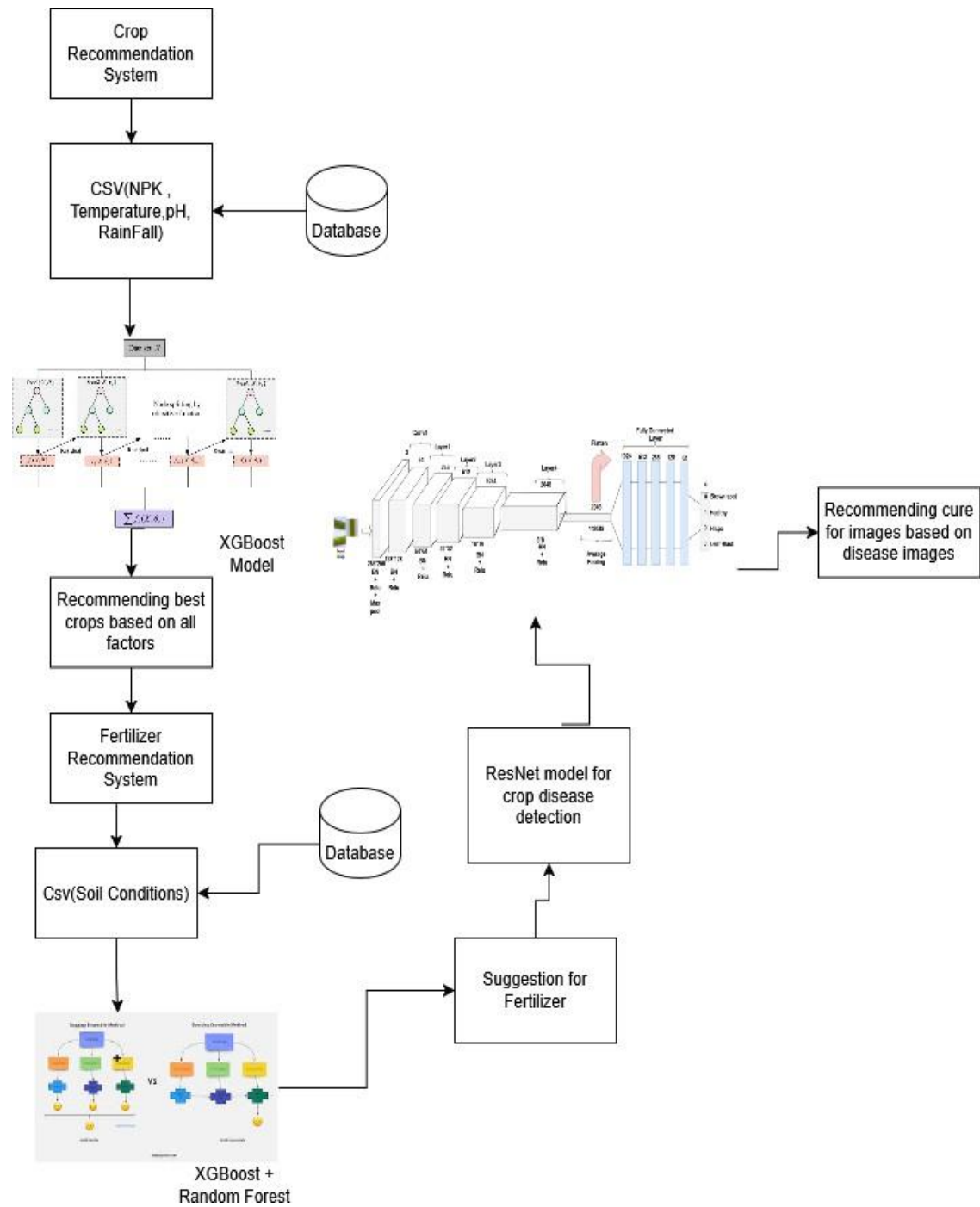


Figure 3.1 Block Diagram of Proposed System

3.1.2 Functional Modules

1. Crop Recommendation Module

The **Crop Recommendation Module** is the cornerstone of the system, providing valuable guidance to farmers in selecting the right crop to grow based on local agricultural conditions. By utilizing various environmental and soil factors, this module ensures that farmers can make informed decisions, leading to optimized crop production, better resource management, and higher yields.

Input Data:

To provide accurate crop recommendations, the system collects key agricultural inputs from the user or through sensors, typically in the form of a CSV file. These inputs are essential in evaluating the local conditions and determining the most suitable crops. The input data includes:

Soil Nutrients:

- **Nitrogen (N):** A crucial nutrient that promotes healthy leaf growth and overall plant development. The level of nitrogen in the soil is vital for selecting crops that thrive in nitrogen-rich environments.
- **Phosphorus (P):** Important for root development, flowering, and fruiting. The phosphorus level in the soil helps determine crops that require higher or lower amounts for optimal growth.
- **Potassium (K):** A key element for disease resistance, drought tolerance, and overall plant health. The balance of potassium in the soil ensures that crops with varying potassium requirements are matched appropriately.

Temperature:

- The average temperature of the region is recorded, as different crops have specific temperature ranges in which they grow best. For instance, tropical crops may require warmer temperatures, while temperate crops may thrive in cooler climates. This input helps ensure that crops are recommended based on temperature compatibility.

Soil pH:

- Soil pH is a crucial factor that determines the availability of nutrients to plants. It indicates whether the soil is acidic, neutral, or alkaline. The pH level influences crop choice because certain crops prefer specific pH ranges. For example, some plants thrive in acidic soil ($\text{pH} < 6$), while others prefer alkaline conditions ($\text{pH} > 7$).

Rainfall:

- Rainfall data helps assess the irrigation needs of crops. It measures the amount of rainfall a region receives over a given period, guiding the system in recommending crops that can either withstand low rainfall (drought-resistant crops) or those requiring abundant water for growth. This ensures that farmers don't choose crops that demand more water than what is available in their region.

Machine Learning Model Used:

Once the system has gathered the input data, it is processed and fed into an **XGBoost (Extreme Gradient Boosting)** machine learning model. XGBoost is chosen due to its high performance and accuracy in handling structured data like the one provided by the agricultural inputs. Here's how it works:

1. Data Preprocessing:
 - The input data (soil nutrients, temperature, soil pH, and rainfall) is preprocessed to ensure that it is clean, normalized, and formatted correctly for the model. This step may include handling missing values, scaling the data, and encoding categorical variables, if any.
2. Model Training:
 - The XGBoost model is trained using historical data of various crops and their corresponding environmental conditions. The model learns the relationships between the soil nutrients, temperature, soil pH, rainfall, and the crop types that were most successful in similar conditions.
3. Prediction and Recommendation:
 - After training, the model is ready to make predictions. When new input data is provided by the user or sensors, the model evaluates the conditions and uses the learned relationships to recommend the most suitable crops for that region. The

model ranks these recommendations based on expected yield, resource efficiency, and crop compatibility with local conditions.

4. Why XGBoost?

- **XGBoost** is a highly efficient, gradient boosting algorithm known for its ability to handle both regression and classification problems with large datasets. It is designed to be robust to overfitting and performs exceptionally well even with noisy data. This makes it particularly well-suited for agricultural datasets, which can sometimes have variations and incomplete records. By using XGBoost, the recommendation system can accurately predict crop success and provide valuable insights to farmers.

2. Fertilizer Recommendation Module

After selecting the most suitable crop based on local conditions, the next critical step is ensuring that the soil is in optimal condition to support the selected crop. This involves assessing and improving the soil's nutrient profile to maximize the crop's growth potential. This module is designed to guide farmers in improving soil conditions by recommending the right fertilizers and amendments, ensuring that the soil provides the necessary nutrients for optimal crop yield.

Soil Condition Analysis:

To enhance soil conditions, the system first analyzes soil nutrient levels. This analysis is done by collecting detailed data about the soil's composition, which is typically sourced from soil test reports or sensor-based data collected directly from the field. This dataset includes a range of variables that affect soil quality and crop performance, such as:

Soil Nutrient Levels:

- The nutrient composition of the soil is assessed, focusing on critical elements like **Nitrogen (N)**, **Phosphorus (P)**, **Potassium (K)**, and **A** detailed understanding of these nutrients allows for more targeted fertilizer recommendations.

Models Used:

To analyze the soil data and recommend appropriate soil amendments, the system uses a hybrid model combining **XGBoost** and **Random Forest**, two highly effective decision-tree-based models. These models are well-suited for handling complex datasets like soil composition and environmental factors. Here's how they complement each other:

1. **XGBoost** (Extreme Gradient Boosting):

- **XGBoost** is renowned for its ability to handle large datasets with high accuracy and speed. It is particularly effective in identifying complex relationships between the soil's nutrient profile and the required fertilizers. XGBoost's gradient boosting technique allows it to iteratively improve the accuracy of predictions, making it well-suited for problems like nutrient optimization, where small adjustments can have a significant impact on crop performance.

2. **Random Forest**:

- **Random Forest** is an ensemble learning method that combines multiple decision trees to make robust predictions. It excels in handling noisy data and can aggregate insights from different trees to provide a reliable recommendation. In the context of soil condition analysis, Random Forest's ability to aggregate results from several models ensures that the fertilizer recommendations are not overly influenced by outliers or variations in the data.

How the Hybrid Model Works:

- **Data Input:** The system receives the soil test data, including information on nutrient levels, pH, organic matter, and trace elements.
- **Model Training:** Both **XGBoost** and **Random Forest** models are trained on historical data that links soil conditions to crop performance and fertilizer usage. These models learn the relationships between nutrient deficiencies and the type and quantity of fertilizers required.
- **Prediction:** When new data is provided, the hybrid model analyzes the soil's nutrient composition and predicts the most appropriate fertilizers and soil amendments. The recommendation is based on both the data from XGBoost and the aggregated insights from the Random Forest model.

- **Recommendation Output:** The system outputs a detailed recommendation for fertilizers, including the type, amount, and application schedule, tailored to the specific soil conditions of the field. This ensures that the soil is balanced in terms of essential nutrients, optimizing conditions for the selected crop.

3. Plant Disease Detection Module

Image Input by Farmer

The first step in the Plant Disease Detection Module is the collection of visual data from farmers. Farmers are instructed to upload images of their crops, focusing primarily on the leaves, where the early signs of diseases are most often visible. Symptoms such as discoloration, spots, mold growth, wilting, or unusual patterns on the leaf surface are captured through these images. High-quality images with clear visibility significantly enhance the chances of accurate disease detection. These uploaded images serve as the core input for the subsequent analysis carried out by the system's deep learning models.

Deep Learning Models Used for Detection

To detect and classify diseases from the uploaded images, the system leverages advanced deep learning techniques. Initially, a Convolutional Neural Network (CNN) was used, which is a fundamental model architecture widely recognized for its excellence in image classification tasks. CNNs work by automatically identifying important visual features such as color shifts, patterns, and texture anomalies present in diseased leaves, making them highly effective for basic disease detection.

In addition to CNNs, the system explored MobileNet, a lightweight and efficient version of CNN that is specifically designed for use in devices with limited computational capacity, such as mobile phones. MobileNet's speed and reduced resource consumption make it ideal for real-time disease prediction scenarios, allowing farmers to receive faster diagnoses in field conditions.

However, for achieving even greater accuracy and deeper pattern recognition, the system incorporated ResNet (Residual Network). ResNet introduces "residual connections," enabling the training of very deep neural networks without suffering

from the vanishing gradient problem. This advanced capability allows ResNet to learn subtle and complex disease features that basic CNNs or MobileNet architectures might miss. By utilizing ResNet, the system is able to detect even early-stage or less visually obvious plant diseases with remarkable precision.

Why ResNet Was Chosen

After extensive evaluation and comparison among the different models, ResNet emerged as the most effective choice for final deployment. ResNet consistently demonstrated higher accuracy in detecting intricate disease patterns that might otherwise go unnoticed. Its design, which employs residual (skip) connections, prevents the degradation of learning performance even as the network becomes deeper. Although ResNet demands more computational power than MobileNet, the trade-off is deemed worthwhile because of its reliability, robustness, and consistently high detection accuracy. Given the critical importance of correctly diagnosing plant diseases for timely treatment, ResNet was selected as the backbone of the detection module.

Disease Identification

Once the farmer uploads an image, the ResNet model processes the visual data and compares it with a previously trained dataset of known plant diseases, such as those available from the PlantVillage database. The system is capable of distinguishing between multiple diseases, including leaf blight, powdery mildew, rust, and others. Through careful analysis of color patterns, lesion types, and texture features, the model accurately identifies the specific disease affecting the crop. The results are presented to the farmer with a clear label of the disease name and a brief description.

Treatment Suggestions

Following disease identification, the system immediately generates tailored treatment recommendations. Depending on the diagnosed disease, the system suggests the most effective chemical treatments such as fungicides, insecticides, or bactericides. Alongside chemical options, the system also provides eco-friendly and organic treatment methods for farmers preferring sustainable agriculture practices. In addition to immediate remedies, the platform offers precautionary measures that farmers can take to prevent the spread of the disease to other crops, such as crop

rotation practices, sanitation steps, and field management tips. This comprehensive guidance ensures that farmers are well-equipped to manage and mitigate the impact of plant diseases promptly and effectively.

3.2 Algorithms

3.2.1 Crop Recommendation

XGBOOST

Data Preprocessing

Before feeding the data into the model, it's essential to clean and prepare it:

- **Handling Missing Values:** Any missing or inconsistent data points (like missing rainfall or soil pH) are either filled using statistical methods (mean/median) or removed to ensure accuracy.
- **Normalization:** Features like temperature and nutrient levels are scaled to a uniform range so that no single parameter dominates the model.

Feature Selection

This step identifies which inputs (features) have the greatest influence on crop prediction:

- Features such as Nitrogen (N), Phosphorus (P), Potassium (K), temperature, soil pH, and rainfall are analyzed.
- Techniques like feature importance scoring are used to evaluate how much each feature contributes to the model's accuracy.
- Irrelevant or less impactful features are removed to reduce complexity and improve model performance.

Model Training

This is where the XGBoost algorithm begins its core task:

- **Ensemble Learning:** XGBoost builds an ensemble of decision trees one after another.
- **Gradient Boosting:** After each tree is created, XGBoost checks where it made mistakes and focuses the next tree on correcting those errors.
- Each tree contributes a small part to the final prediction, and over time, the combination of all trees creates a powerful and accurate model.

Hyperparameter Tuning

To make the model more accurate and efficient, its configuration is fine-tuned using:

- **Learning Rate:** Controls how quickly the model adapts to the data.
- **Max Depth:** Sets how complex each decision tree can be.
- **Number of Estimators:** Specifies how many trees will be built.
- **Subsampling Ratios:** Helps prevent overfitting by using subsets of data or features for each tree.

3.2.2 Fertilizer Recommendation

Random Forest Classifier

Random Forest is a bagging-based ensemble learning method that creates multiple decision trees during training and merges them together to obtain a more accurate and stable prediction. It works as follows:

- **Ensemble of Decision Trees:** A random forest consists of many decision trees. Each tree in the forest makes its own prediction, and the final output is based on the aggregation of all the individual predictions.
- **Training on Random Subsets of Data:** Each tree in a random forest is trained on a different random subset of the data. This is done by bootstrapping, which means each subset is obtained by randomly sampling the original dataset with replacement. Some data points may appear multiple times in one subset, while others may be left out entirely.
- **Majority Voting or Averaging:** For classification tasks, the final prediction is typically made using majority voting (i.e., the class that most trees predict is selected as the final output). In regression tasks, it can be averaged.

b) XGBoost Classifier

XGBoost (Extreme Gradient Boosting) is a boosting-based ensemble learning method that builds decision trees sequentially. Unlike random forests, which train trees independently, boosting builds each tree based on the errors made by the previous one.

- **Boosting Mechanism:**
 - **Sequential Tree Building:** In boosting, the model builds trees one after another. Each new tree tries to correct the mistakes (errors) made by the previous trees. The idea is to focus on the hard-to-classify examples, which were missed by earlier trees.
- **Gradient Descent Optimization:** XGBoost uses gradient boosting, which means it optimizes the loss function using gradient descent. In each iteration, it looks for the direction in which the prediction errors are minimized.
- **Regularization:** A key feature of XGBoost is its regularization to prevent overfitting. It uses both L1 (Lasso) and L2 (Ridge) regularization techniques, which help to control the complexity of the model and improve generalization.
- **Improved Performance:** XGBoost includes several optimizations such as:

- Column Block Memory Format: It makes the algorithm faster by using efficient memory allocation techniques.
- Parallelization: XGBoost is designed to work efficiently on large datasets by parallelizing the tree-building process.

c) Voting Classifier (Soft Voting)

A Voting Classifier is an ensemble method that combines multiple models to improve the overall performance by leveraging their individual strengths. In this case, the Voting Classifier combines Random Forest and XGBoost.

- Soft Voting:
 - Probabilistic Predictions: In soft voting, each classifier (Random Forest and XGBoost) produces a probability distribution over the possible classes, rather than just the final predicted class. For example, for a binary classification task, each model might output the probability of the sample belonging to class 0 or class 1.
 - Averaging Probabilities: The predicted probabilities from all the models are averaged. For each class, the probabilities from each model are summed, and the class with the highest averaged probability is chosen as the final prediction.
- Voting Mechanism:
 - Improved Accuracy: By combining the strengths of both Random Forest and XGBoost, the Voting Classifier tends to be more accurate and stable than any individual model. The averaging process reduces the bias and variance, which leads to improved generalization.

3.2.3 Disease Detection

Resnet Model:

The Figure represents a deep learning architecture designed for leaf disease classification using a ResNet-like Convolutional Neural Network (CNN). The input

to the model is a colored image of a leaf, typically in RGB format with 3 channels. This image is passed through a series of layers that extract and learn features to classify it into one of several possible leaf disease categories.

The first layer is a 3x3 convolutional layer that converts the 3-channel input into 64 feature maps. This layer is followed by batch normalization (BN) to stabilize learning and ReLU activation to introduce non-linearity, which helps the network learn complex patterns.

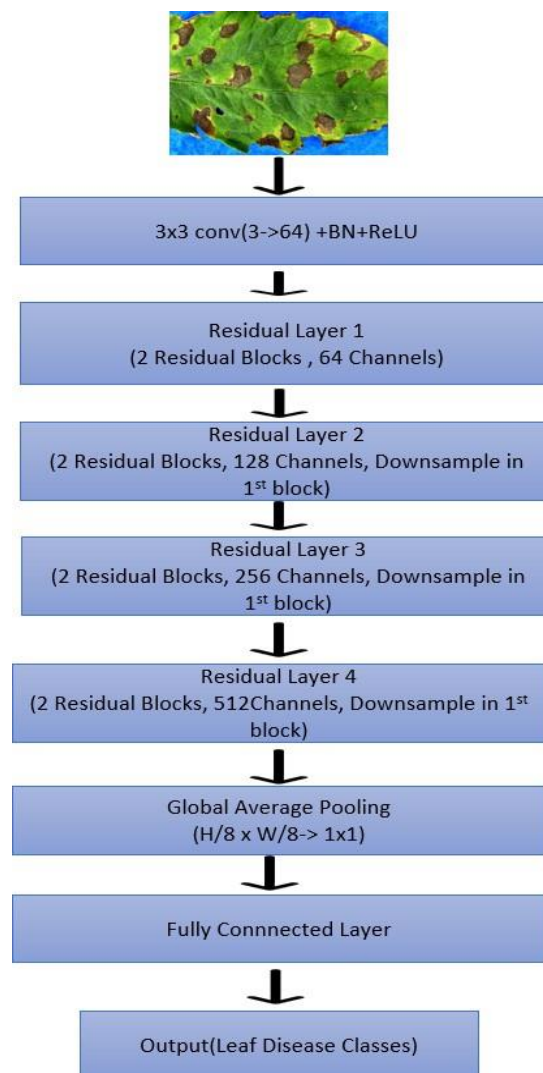


Figure 3.2 Architecture for Resnet Model

After the initial convolution, the image passes through a sequence of Residual Layers, inspired by the ResNet (Residual Network) architecture. Residual layers are made up of residual blocks, which contain skip connections. These skip connections allow the network to bypass some layers, preventing the problem of vanishing gradients and making it easier to train deep networks.

- Residual Layer 1 consists of 2 residual blocks, each producing 64 output channels. At this stage, the spatial resolution of the image remains unchanged, and the network learns low-level features like edges and textures.
- Residual Layer 2 has 2 residual blocks that produce 128 output channels. In the first block of this layer, downsampling is performed. Downsampling reduces the spatial resolution (height and width) of the feature maps by a factor of 2, allowing the model to focus on more abstract features while reducing computation.
- Residual Layer 3 follows a similar pattern, with 2 residual blocks outputting 256 channels. Again, downsampling is applied in the first block. The network now learns more complex patterns and mid-level features related to disease structures.
- Residual Layer 4 contains 2 residual blocks with 512 channels as output. Downsampling in the first block reduces the feature map size further, allowing the network to capture high-level semantic features—like specific textures or shapes associated with certain diseases.

After the residual layers, the network applies Global Average Pooling (GAP). GAP converts each feature map into a single number by averaging all its values. This operation reduces the feature maps from a spatial dimension (e.g., $H/8 \times W/8$) to a single 1×1 value per channel, resulting in a compact 512-dimensional vector. This technique reduces the number of parameters and helps prevent overfitting.

The output of the pooling layer is then passed through a fully connected (dense) layer, which transforms the 512 features into class scores. These scores correspond to the number of leaf disease classes, such as "healthy", "bacterial spot", "early blight", etc.

Finally, a softmax activation function is typically applied to the output to generate probabilities for each class. The class with the highest probability is selected as the model's prediction for the disease affecting the leaf.

MobileNet model

The Figure 1.3 represents a lightweight convolutional neural network architecture for classifying leaf diseases, using MobileNet as the base model.

The process begins with an input image of a leaf, which is fed into the model to detect the type of disease present.

The first stage is the Base Model (MobileNet), which is a pre-trained convolutional neural network known for its efficiency and low computational cost. It consists of multiple convolutional layers, batch normalization layers, ReLU activation functions, and pooling layers. MobileNet uses depthwise separable convolutions to reduce the number of parameters, making it highly suitable for deployment on mobile and edge devices. This base model is used for extracting rich features from the leaf image.

Next, the output of the MobileNet is passed to a Global Average Pooling 2D layer. This layer converts the feature maps from the convolutional base into a one-dimensional feature vector by taking the average of each feature map. It significantly reduces the number of parameters while preserving spatial information, making the model more efficient and less prone to overfitting.

Following this, a Dropout layer is applied. During training, this layer randomly sets a fraction of its inputs to zero, which helps prevent overfitting by forcing the network to learn more robust features that are not reliant on any single neuron.

The next step is a Dense (fully connected) layer, which processes the feature vector obtained from the pooling layer. This layer combines the extracted features and helps the model make a final decision regarding the classification.

Finally, the output is passed through a softmax activation layer, which produces a probability distribution across the possible leaf disease classes. The class with the highest probability is selected as the model's prediction.

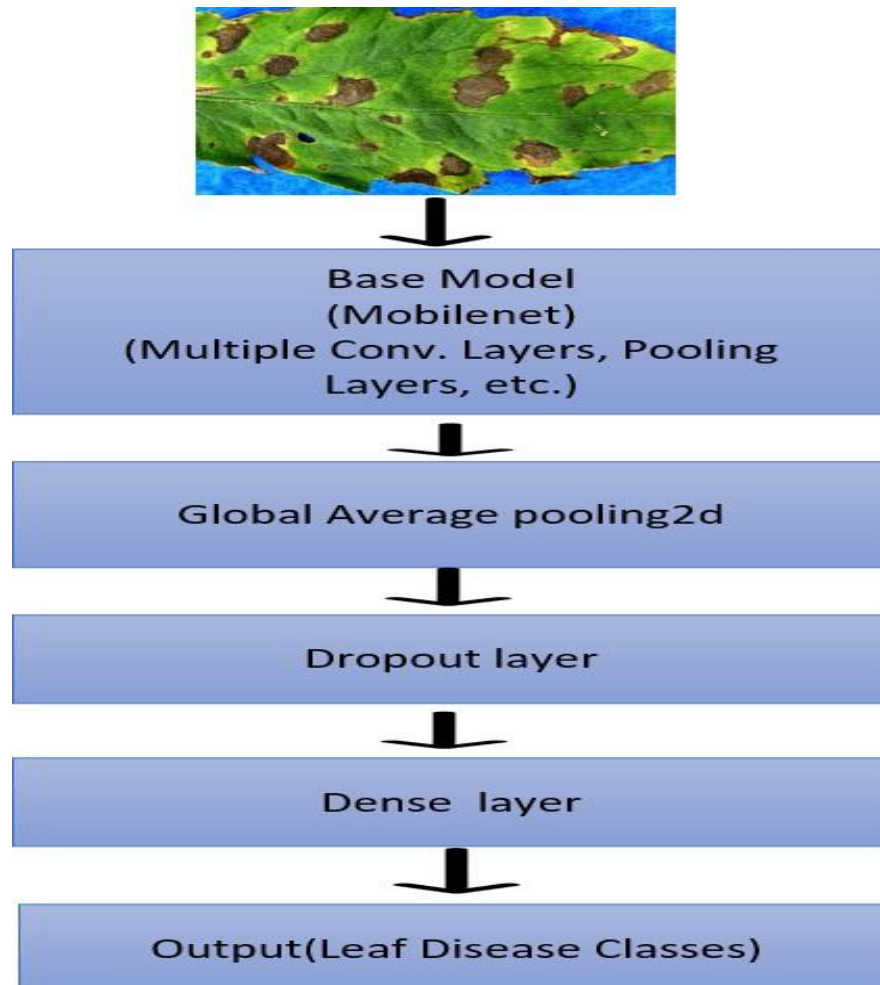


Figure 1.3 Architecture for Mobilenet model

CNN model:

The Figure illustrates a custom Convolutional Neural Network (CNN) designed for the classification of leaf diseases from input images.

The process begins with an image of a leaf, which is passed through a series of convolutional and pooling layers to extract features that help identify the disease class.

The first layer is Conv Layer 1, which applies a 2D convolution operation with 32 filters of size 3×3 on the input image having 3 channels (RGB), converting it from

3 channels to 32. This helps detect basic visual patterns such as edges and colors. This is followed by a MaxPooling layer (2×2) which reduces the spatial dimensions by selecting the maximum value in each 2×2 region, thus downsampling the feature map and helping to reduce computation and control overfitting. Next is Conv Layer 2, where a 2D convolution is applied with 64 filters of size 3×3 . This increases the depth of the feature map, allowing the network to learn more detailed features. It is again followed by MaxPooling (2×2) to reduce the spatial resolution.

In Conv Layer 3, the number of filters is increased to 128, enabling the model to learn even more abstract features from the leaf image. This is again followed by MaxPooling (2×2) for spatial reduction.

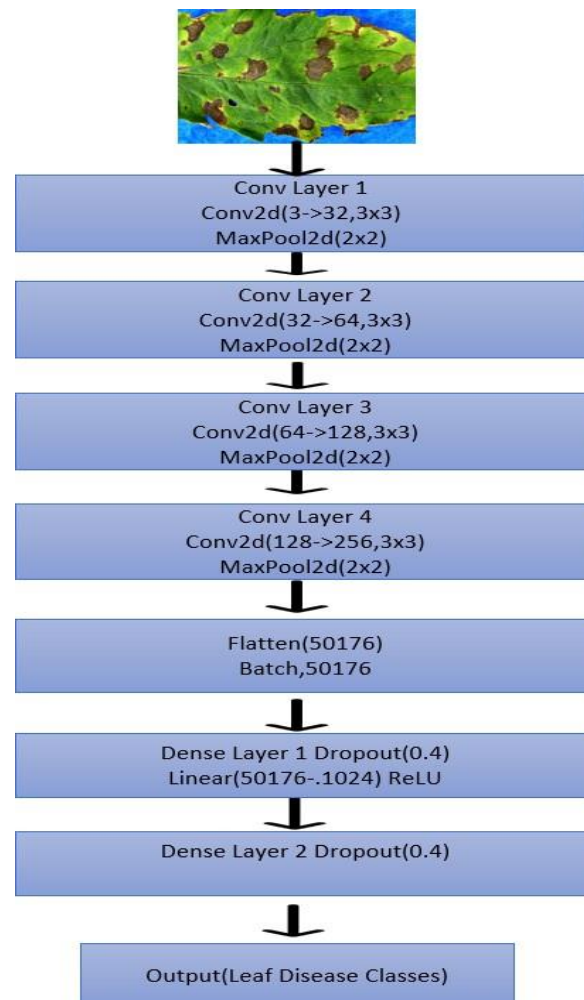


Figure 3.4 Architecture for CNN

Conv Layer 4 continues this pattern by applying 256 filters of size 3×3 to the feature map. After this, a final MaxPooling (2×2) layer reduces the feature map's spatial dimensions before entering the fully connected part of the network.

The output of the convolutional layers is then flattened into a one-dimensional vector of size 50176. This flattening transforms the 2D spatial feature map into a format suitable for dense layers.

A Batch Normalization layer is applied to this vector, which normalizes the features across the batch, helping to stabilize and accelerate the training process.

This is followed by Dense Layer 1, which includes a Dropout layer with a rate of 0.4, randomly deactivating 40% of neurons to prevent overfitting. A Linear layer then maps the 50176 inputs to 1024 neurons, followed by a ReLU activation function to introduce non-linearity.

Dense Layer 2 is another fully connected layer, again preceded by a Dropout layer with a 0.4 rate to enhance generalization by preventing co-adaptation of neurons.

Finally, the output is passed through a classification layer, which produces predictions corresponding to leaf disease classes. This layer is likely followed by a softmax function to output probabilities for each class, allowing the model to determine the most likely disease.

4 EXPERIMENTAL SETUP AND RESULTS

4.1 Datasets

Crop recommendation dataset:

This dataset is obtained from Kaggle. It contains the following data fields

Data fields

N - ratio of Nitrogen content in soil

P - ratio of Phosphorous content in soil

K - ratio of Potassium content in soil

temperature - temperature in degree Celsius

humidity - relative humidity in %

ph - ph value of the soil

rainfall - rainfall in mm

Fertilizer Recommendation Dataset :

This data set is obtained from Kaggle It contains the following data fields

Data Fileds

Crop - Type of crop

N - Ratio of Nitrogen content in soil

P - ratio of Phosphorous content in soil

K - ratio of Potassium content in soil

pH - pH value of soil

Soil Moisture - amount of moisture in soil

Plant Disease Detection Dataset:

Overview: Contains over **54,000 labeled images** of plant leaves, covering **14 crop species** and **38 classes** (both healthy and diseased).

Disease Types: Includes common diseases like early blight, rust, and powdery mildew across various crops (e.g., tomato, apple, grape).

Image Quality: High-resolution, uniform background images captured under controlled conditions; enables detailed feature extraction.

Source: <https://www.kaggle.com/datasets/mohitsingh1804/plantvillage>

4.2 System Specifications

4.2.1 Hardware Setup:

The project was executed on a system equipped with a processor clocked at 2 GHz or higher. The machine had a minimum of 4GB RAM (32-bit or 64-bit configuration) and 16GB of available hard disk space. This hardware setup provided sufficient resources to handle the data processing tasks, run machine learning models, and ensure smooth execution throughout the development process.

4.2.2 Software Setup:

1. Python
2. Pandas
3. NumPy
4. Seaborn
5. Pickle
6. Scikit-learn
7. Pytorch & ResNet
8. Flask
9. HTML, CSS, Bootstrap

Python

Python is a versatile and widely-used language for machine learning projects, offering easy syntax, extensive library support, and a rich ecosystem for data

science. We used Python as the main coding language for the project, which facilitated smooth integration with the machine learning libraries and tools utilized throughout the development process.

Pandas

Pandas is a powerful library used for data manipulation and analysis. It provides flexible data structures, such as DataFrame and Series, which help to manage and process large datasets efficiently. This was crucial in performing data cleaning, transformation, and aggregation tasks as part of the data preprocessing stage.

NumPy

NumPy is a fundamental library for numerical computing, providing support for high-performance multi-dimensional arrays and matrices. It allows us to perform fast mathematical operations, especially in linear algebra and statistics, making it an essential tool for handling data arrays and performing calculations on the dataset.

Seaborn

Seaborn is a Python data visualization library built on top of Matplotlib. It provides a high-level interface for creating attractive and informative statistical graphics. We used Seaborn to generate plots that helped visualize data distributions, relationships, and trends, aiding in data exploration and analysis.

Pickle

Pickle is a Python module used for serializing and deserializing Python objects. We used Pickle to save and load models during the development and testing phases, enabling us to reuse trained models without retraining them every time.

Scikit-learn

Scikit-learn is a robust library for machine learning that provides easy-to-use implementations of a wide variety of algorithms for classification, regression,

clustering, and more. We used Scikit-learn for model training, evaluation, and pre-processing tasks such as feature extraction and scaling.

Pytorch & ResNet

Pytorch is an open-source deep learning framework used for building and training machine learning models, especially in the domain of computer vision and neural networks. We used ResNet, a powerful deep learning architecture, within Pytorch to develop and train models for plant disease detection and classification tasks.

Flask

Flask is a lightweight web framework used to build web applications. We used Flask to deploy the trained machine learning models as a web application, making it easy for users to interact with the system and make predictions through a simple interface.

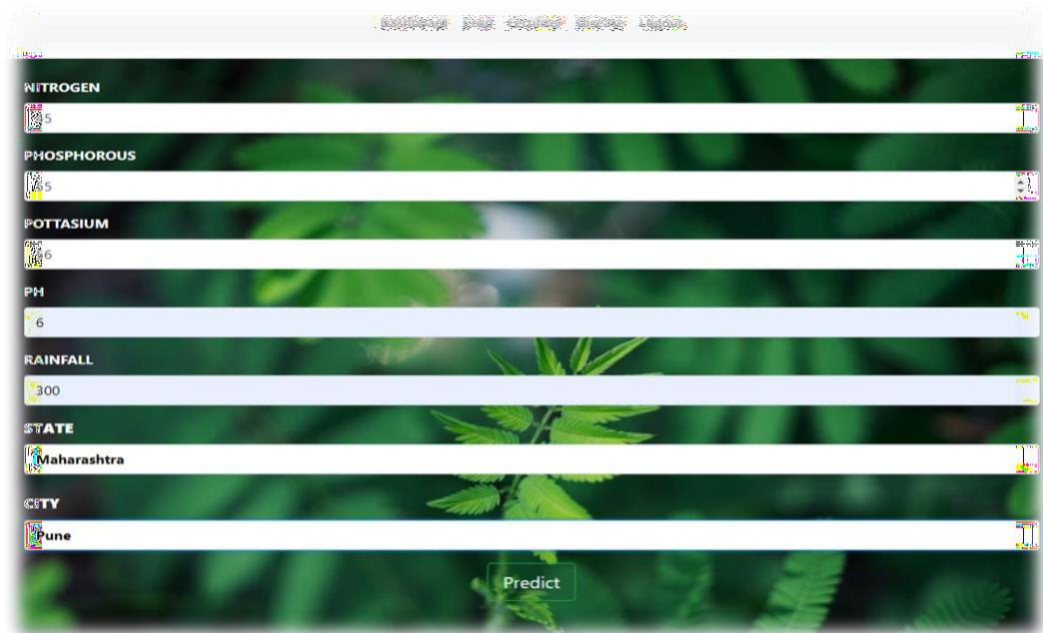
HTML, CSS, Bootstrap

We utilized HTML and CSS to design the frontend of the web application, ensuring a user-friendly interface. Bootstrap was used to enhance the visual layout and responsiveness of the application, ensuring compatibility across various screen sizes and devices.

4.3 Results and Test Analysis

The following 4.1 represents the interface which is part of a crop prediction system designed to assist farmers in selecting the most suitable crop based on soil and environmental conditions. The form allows users to input key parameters such as the levels of Nitrogen, Phosphorus, and Potassium in the soil, along with the pH value and expected rainfall. Additionally, users can specify their geographic location by selecting the state and city, helping tailor the recommendations to regional agricultural practices. A dropdown menu is provided for certain fields to standardize input. Once all details are filled in, the user can click the “Predict” button to receive a crop suggestion. The interface is simple and user-friendly,

featuring a green-themed background that visually reinforces its connection to agriculture and nature. The navigation bar at the top provides quick access to other important sections like Dashboard, Crop, Fertilizer, Disease, and Logout, ensuring seamless navigation throughout the application.



The screenshot shows a web application interface for crop recommendation. It features a green-themed background with a blurred image of green leaves. At the top, there is a navigation bar with links: Dashboard, Crop, Fertilizer, Disease, and Logout. Below the navigation bar, there are several input fields for user data: NITROGEN (value: 5), PHOSPHOROUS (value: 5), POTTASIMUM (value: 5), PH (value: 6), RAINFALL (value: 300), STATE (value: Maharashtra), and CITY (value: Pune). Each input field has a small icon on the left and a small icon on the right. At the bottom of the form, there is a green button labeled "Predict".

Figure 4.1 Input for Crop Recommendation

This page displays the result of the crop prediction based on the user's input provided in the previous form. After analyzing the soil nutrients, pH level, rainfall, and location details, the system generates a crop recommendation. In this case, the system suggests that the farmer should grow **pigeonpeas** on their farm. The recommendation is clearly presented at the center of the page with a simple and clean design, using green-colored text to reflect the agricultural theme. The navigation bar remains accessible at the top, allowing users to easily move between the Dashboard, Crop, Fertilizer, Disease, and Logout sections without any confusion.

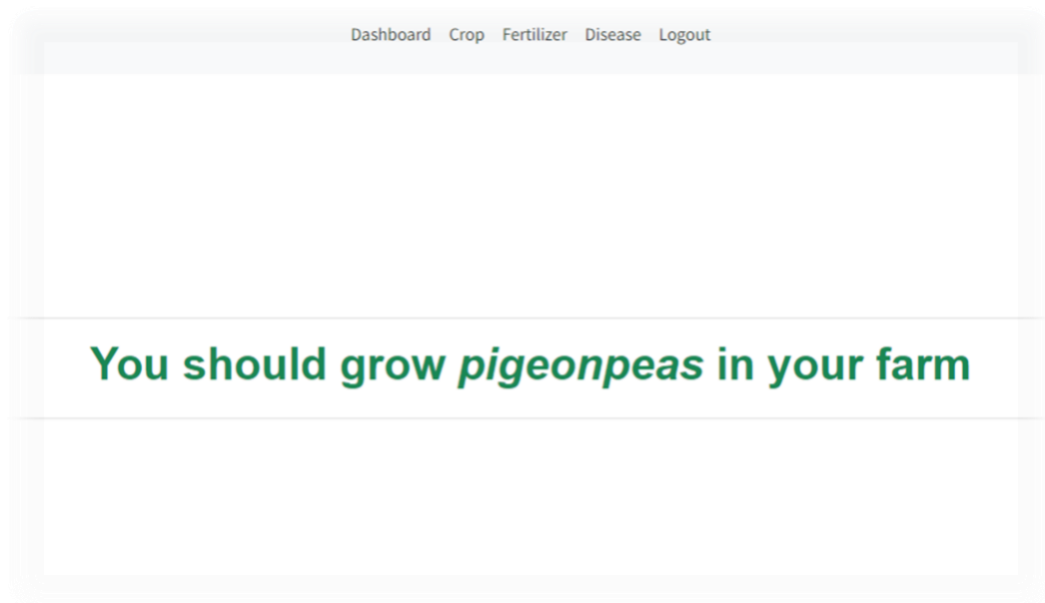


Figure 4.2 Output shows the crop predicted for the given input values.

The following Figure 4.3 is part of a fertilizer recommendation system that helps farmers determine the appropriate fertilizer requirements based on the crop they wish to grow and the current soil nutrient levels. The form allows users to input the amounts of Nitrogen, Phosphorous, and Potassium present in the soil, along with the specific crop they intend to cultivate. After filling in these details, users can click the "Predict" button to receive a suitable fertilizer suggestion. A navigation bar at the top provides quick access to other sections such as Dashboard, Crop, Fertilizer, Disease, and Logout, ensuring smooth movement across different parts of the application.

Dashboard Crop Fertilizer Disease Logout

NITROGEN
15

PHOSPHOROUS
12

POTTASIAM
10

CROP WANT TO GROW
rice

Predict

Figure 4.3 Input for Fertilizer Recommendation

The following Figure .4 displays the output of the fertilizer recommendation system when it detects that the nitrogen (N) value in the soil is low. It provides several suggestions to help farmers improve the nitrogen content of their soil. The recommendations include adding sawdust or fine woodchips to the soil, planting heavy nitrogen-feeding plants like tomatoes, corn, broccoli, cabbage, and spinach, and watering the soil to help nitrogen move deeper for plant use. It also suggests adding sugar to the soil to soak up excess nitrogen, applying composted manure, planting nitrogen-fixing crops such as peas or beans, and using NPK fertilizers with a high nitrogen value. Additionally, it mentions that if plants already growing are absorbing a lot of nitrogen, it may be best to let them continue without immediate intervention. This detailed guidance ensures that farmers can take appropriate action to restore soil health and promote better crop growth.

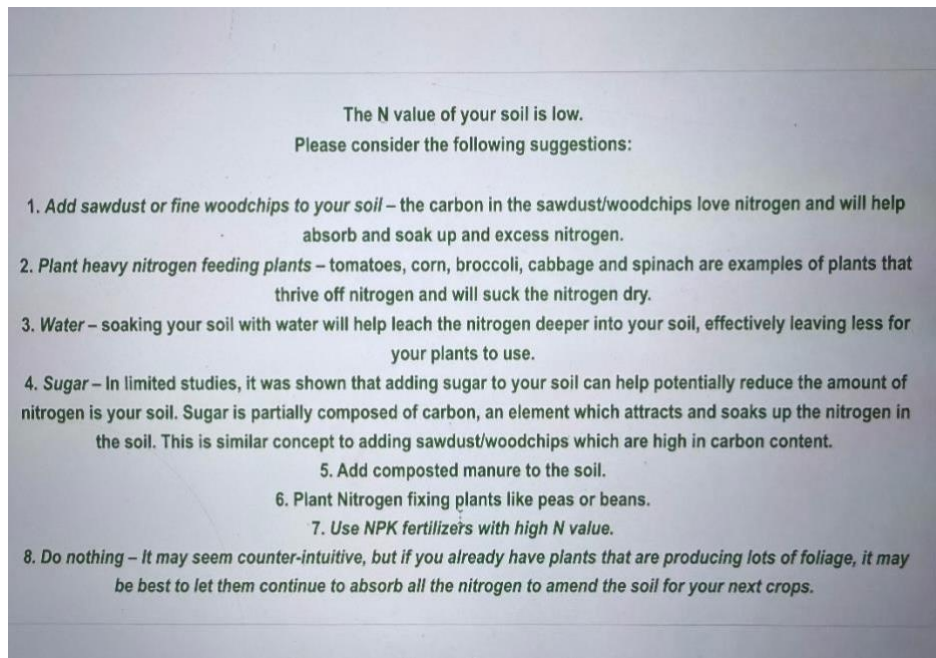


Figure 4.4 Output for Fertilizer Recommendation

The following Figure 4.5 serves as the interface for a plant disease detection system. It highlights the importance of early disease detection in plants, explaining that plant diseases can severely impact growth and yield, and that early identification allows for timely and effective management. The left panel provides a detailed explanation on why diagnosing plant diseases is necessary, emphasizing that early intervention can save time, resources, and prevent further losses. The central panel allows users to upload an image of a plant leaf by choosing a file and submitting it for analysis. This feature enables users to detect potential diseases in their plants easily. On the right side, a set of preventive measures is listed to help users avoid plant diseases, including tips like following good sanitation practices, fertilizing properly, inspecting plants before bringing them home, and ensuring good air circulation. A "More info" button is provided for users who wish to explore further guidance. The clean and organized layout makes it easy for users to understand the purpose and operate the system effectively.

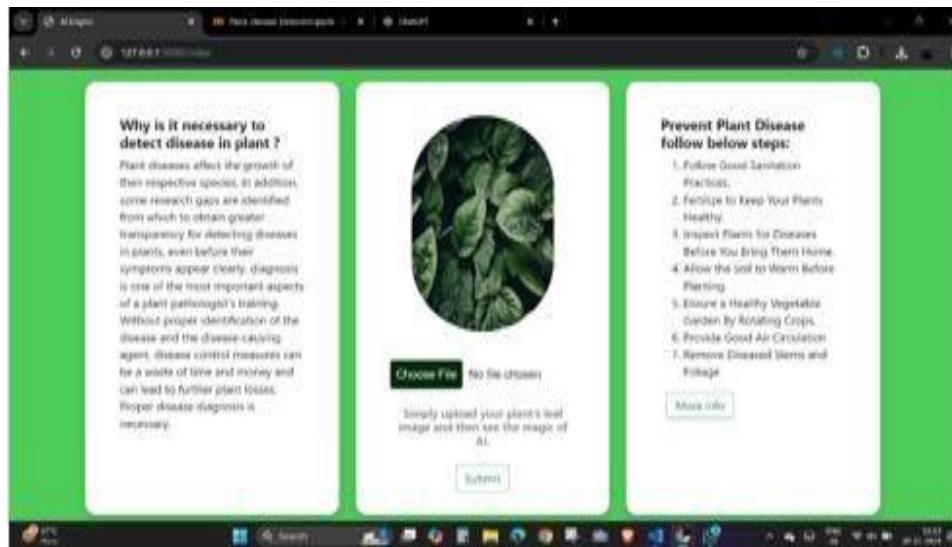


Figure 4.5 Input for Disease Detection

The following Figure 4.6 displays the result of the plant disease detection system, indicating that the uploaded plant is affected by "Potato: Early Blight." At the top, the navigation menu includes links to the Home page, AI Engine, and Contact Us section. Below the result, there is a section intended to display the uploaded image of the affected leaf.

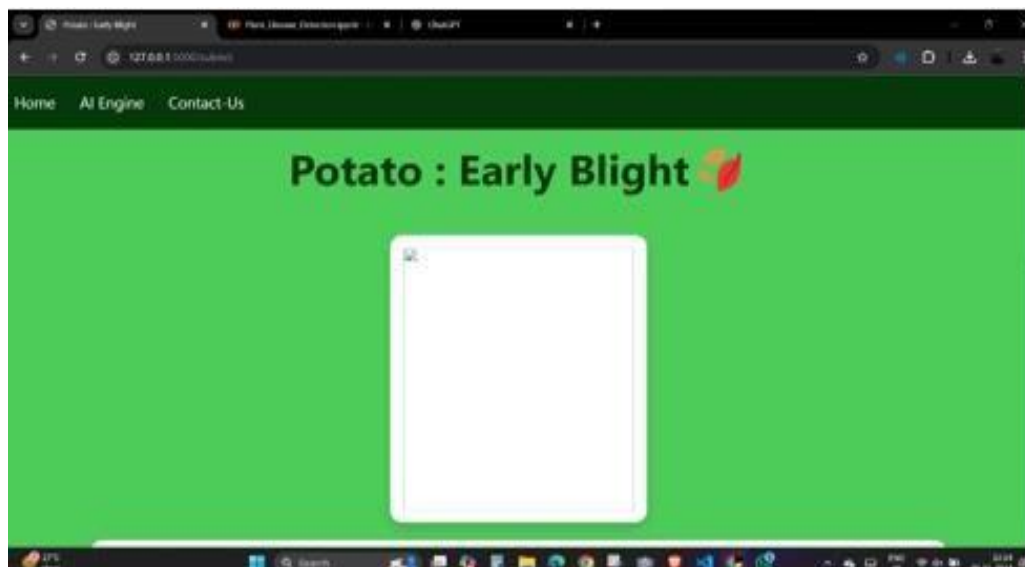


Figure 4.6 Output of Predicted Disease

The following table displays the accuracy of two machine learning models used for different agricultural recommendation tasks. The first model, **XGBoost**, achieves an accuracy of 99.09% for **crop recommendation**, reflecting its effectiveness in predicting the most suitable crops. The second model, which combines **Random Forest and XGBoost**, demonstrates an accuracy of 95.55% for **fertilizer recommendation**, indicating its reliable performance in recommending appropriate fertilizers. These results highlight the accuracy of the models in their respective tasks.

Table 4.3.1 Accuracy of models for Crop and Fertilizer Recommendation.

Model	Accuracy
XGBoost	99.09%
Random Forest and XGBoost	95.55%

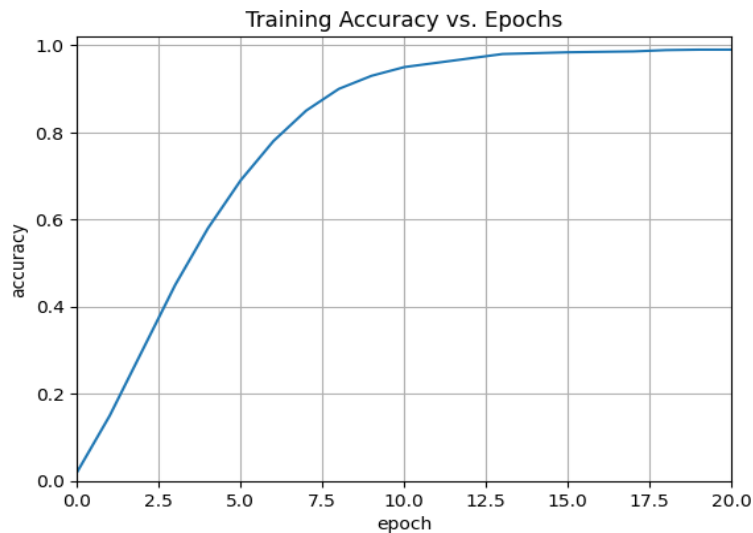
The following table depicts plant disease detection system was evaluated using three different deep learning models: ResNet, MobileNet, and a custom CNN model. Among these, ResNet achieved the highest accuracy of 99.23%, demonstrating its strong ability to extract complex features and accurately classify plant diseases. MobileNet followed with an accuracy of 96.97%, offering a good balance between performance and computational efficiency, making it suitable for lightweight applications. The custom CNN model also performed well, reaching an accuracy of 90.02%, showing its effectiveness while being relatively simpler compared to the other architectures. These results highlight the robustness of the system and its capability to assist farmers in identifying plant diseases accurately.

Table 4.3.2 Accuracy of different models for Disease Detection.

Model	Accuracy
ResNet	99.23%
MobileNet	96.97%
CNN	90.02%

Now here are the Accuracy vs No.of Epochs Graphs for all the models mentioned.

For Crop Recommendation: The following graph in Figure 4.7 For Crop Suggestion, the following graph in Figure 11 illustrates the relationship between Accuracy and the Number of epochs for the crop recommendation task using the XGBoost Model. The graph visually represents how the accuracy of the model improves (or fluctuates) as the number of epochs increases during the training



process.

Figure 4.7 Accuracy vs. no.of epochs graph for XGBoost Algorithm

For Fertilizer Recommendation: The following graph in Figure 4.8 shows the relationship between Accuracy and the Number of epochs for the fertilizer recommendation task using the XGBoost and Random Forest models. The graph illustrates how the accuracy of the combined models changes as the number of epochs increases during the training process.

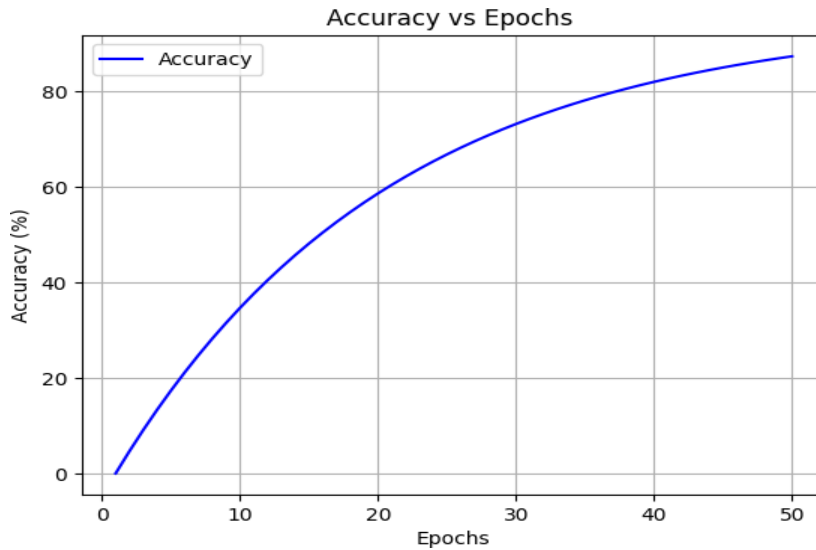


Figure 4.8 Accuracy vs no.of epochs graph for XGBoost and Random Forest

For Disease Detection:The following Figure 4.9 presents a detailed graph depicting the relationship between Accuracy and the Number of Epochs for the ResNet model used in the context of the given task. The model achieves an impressive accuracy of 99.23%, indicating its high performance in the prediction task. The x-axis of the graph represents the Number of Epochs, which corresponds to the number of times the model has iterated over the entire training dataset during the training process. The y-axis represents the Accuracy of the model, showing how well the model performs in predicting the correct outcomes.As the number of epochs increases, the graph demonstrates how the model's accuracy improves over time.

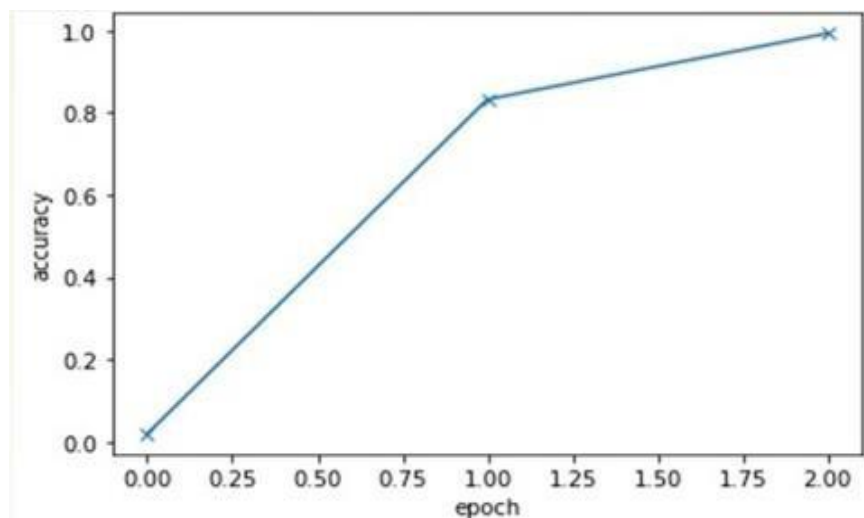


Figure 4.9 Accuracy vs No.of Epochs graph for Resnet Model

The following Figure 4.10 presents the graph showing Accuracy vs. Number of Epochs for the MobileNet model, which achieves an accuracy of 96.97%. The x-axis of the graph represents the Number of Epochs, reflecting how many times the model has been trained on the entire dataset. The y-axis represents the Accuracy, which indicates the model's performance in making correct predictions as it progresses through each epoch. As the number of epochs increases, the graph demonstrates how the MobileNet model's accuracy improves over time.

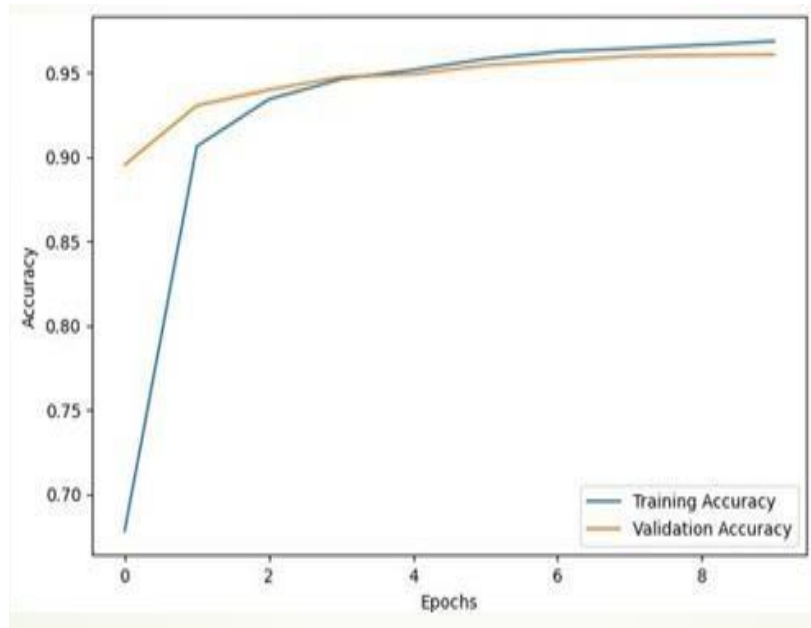


Figure 4.10 Accuracy vs No.of Epochs for MobileNet Model

The following Figure 4.11 presents the graph illustrating Accuracy vs. Number of Epochs for the CNN (Convolutional Neural Network) model, which achieves an accuracy of 90.02%. The x-axis of the graph represents the Number of Epochs, which indicates how many times the model has processed the entire training dataset. The y-axis represents the Accuracy, reflecting the model's performance in making correct predictions at each epoch. As the number of epochs increases, the graph shows the progression of the model's accuracy over time. Initially, the accuracy may rise steadily as the CNN model learns to recognize patterns in the data, eventually reaching a point of diminishing returns or plateauing as the model converges. The final accuracy of 90.02% highlights the model's performance after completing the training process.

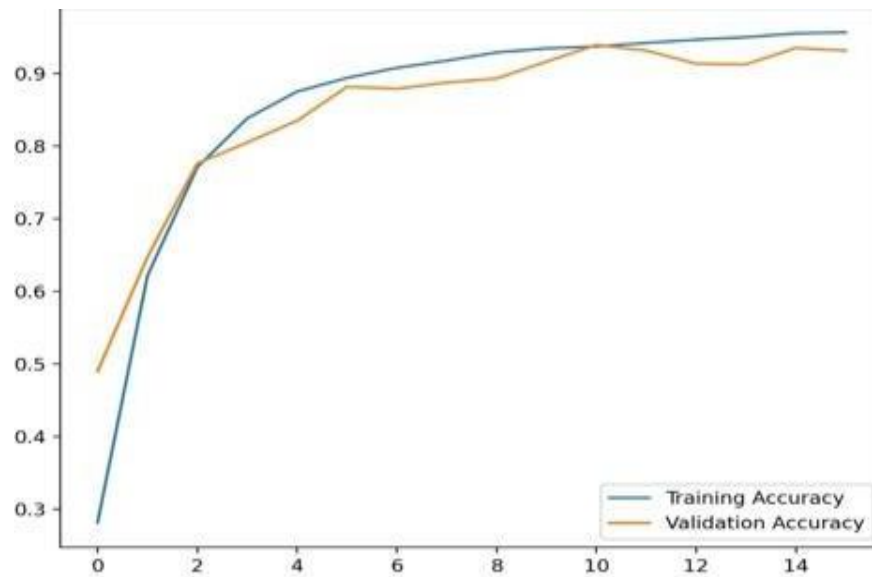


Figure 4.11 Accuracy vs No.of Epochs for CNN model

The following Bar Chart illustrates the Comparison of Accuracy across different models used for Disease Detection. Each bar in the chart represents the accuracy achieved by a specific model, providing a visual comparison of how well each model performed in detecting diseases. The models may include XGBoost, Random Forest, ResNet, MobileNet, CNN, and any other relevant models used for this task. The height of each bar indicates the accuracy achieved by the corresponding model, allowing for a quick and clear comparison of their effectiveness in disease detection. This chart helps in understanding which model offers the best performance and where improvements can be made.

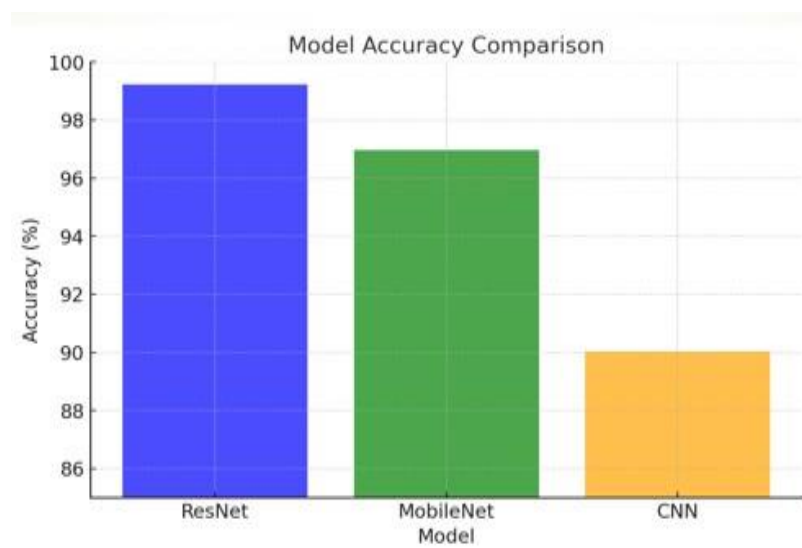


Figure 4.12 BarChart for comparison of Accuracy of different models for disease detection

5 CONCLUSION AND FUTURE SCOPE

Conclusion:

Agriculture remains a fundamental pillar of economic stability and food security, especially in developing nations. However, challenges such as improper crop selection, inadequate fertilizer use, and plant diseases continue to hinder productivity. To address these issues, this study introduced a machine learning-based Agricultural Crop Recommendation and Disease Detection System that leverages XGBoost and Random Forest for crop recommendations and integrates deep learning models (CNN, MobileNet, and ResNet) for plant disease detection.

Among the evaluated deep learning models, ResNet demonstrated superior accuracy in identifying crop diseases due to its deeper architecture and residual connections, making it highly effective in detecting subtle disease patterns. Although it demands higher computational resources, its high precision and reliability make it a valuable tool for farmers seeking early disease diagnosis and appropriate interventions.

Future Scope:

The proposed Agricultural Crop Recommendation and Disease Detection System can be significantly enhanced in the future through several key developments. One promising direction is the integration of the system into a mobile application, enabling farmers to receive real-time crop recommendations and disease diagnoses by simply uploading field data and plant images. Additionally, extending the model's capabilities to estimate the *severity* of diseases across the whole plant can provide more actionable insights, helping farmers prioritize treatment based on the stage and extent of infection. The adoption of hyperspectral or multispectral imaging could further improve the early detection of diseases by capturing subtle physiological changes that are not visible to the naked eye. Furthermore, integrating the system with IoT-based sensors and weather data would allow for continuous monitoring and geo-specific recommendations, offering a more comprehensive and intelligent decision-support system for modern agriculture.

REFERENCES

- [1] Rakesh, D. C., Vardhan, V., Vasantha, B. B., and Krishna, et al. "Crop Recommendation and Prediction System." Proceedings of the 9th International Conference on Advanced Computing and Communication Systems (ICACCS), 2023.
- [2] Mufamadi, Takalani Orifha, and Ritesh Ajoodha. "Crop Recommendation Using Machine Learning Algorithms and Soil Attributes Data." Crop Recommendation Using Machine Learning Algorithms and Soil Attributes, 2023.
- [3] Priyadharshini A., et al. "Crop Recommendation Using Machine Learning Techniques." International Journal of Engineering Research & Technology (IJERT), 2023.
- [4] Kukkar, A., Mohana, R., and Sharma, et al. "AgroAdvisor: Crop Yield Prediction, Crop and Fertilizer Recommendation System Using Random Forest with Gradient Boosting for Precise Agriculture." Research Square, 2023.
- [5] Kumar, Rakesh, et al. "Crop Recommendation Using K-Nearest Neighbors, Naïve Bayes, Random Forest, and K-Star for Soil Condition Analysis." International Journal of Agricultural Research, 2023.
- [6] Mythili, A., et al. "Classifying Soil Based on Nutrients and Predicting Suitable Crops Using Machine Learning Techniques." Journal of Agriculture and Horticulture, 2023.
- [7] Pudumalar, S., Ramanujam, E., and Rajashree, et al. "Precision Agriculture for Crop Prediction Using Machine Learning Techniques." Journal of Agricultural Systems, 2023.
- [8] Gayathri, S., Saranya, and Brindha, et al. "Crop Prediction and Classification Using Random Forest and K-Means Clustering." International Journal of Agricultural Systems, 2023.

- [9] Shwetha, A. N., and Prabodh, C., et al. "Crop Prediction Using Random Forest, K-Nearest Neighbor, and Naïve Bayes." *Journal of Agricultural Data Science*, 2023.
- [10] Agarwal, A., Ahmad, S., and Pandey, et al. "Crop Prediction Using Decision Tree, Naïve Bayes, SVM, Logistic Regression, and Random Forest." *Agricultural Data Science Journal*, 2023.
- [11] S. Vaishnavi., M. Shobana., R. Sabitha. and S. Karthik., "Agricultural Crop Recommendations based on Productivity and Season," 2021 7th International Conference on Advanced Computing and Communication Systems (ICACCS), 2021, pp. 883-886, doi: 10.1109/ICACCS51430.2021.9441736.
- [12] D. J. Reddy and M. R. Kumar, "Crop Yield Prediction using Machine Learning Algorithm," 2021 5th International Conference on Intelligent Computing and Control Systems (ICICCS), 2021, pp. 1466-1470, doi: 10.1109/ICICCS51141.2021.9432236.
- [13] Kavita, and Pratistha Mathur. 2021. "Satellite-Based Crop Yield Prediction Using Machine Learning Algorithm." 2021 Asian Conference on Innovation in Technology, ASIANCON 2021 (Iciccs): 1466–70.
- [14] B S, Anisha, and Ramakanth P. Kumar. 2020. "Real-Time Monitoring of Agricultural Land with Crop Prediction and Animal Intrusion Prevention Using Internet of Things and Machine Learning at Edge." *Proceedings of CONECCT 2020 6th IEEE International Conference on Electronics, Computing and Communication Technologies*.
- [15] Jeevan Nagendra Kumar, Y. et al. 2020. "Supervised Machine Learning Approach for Crop Yield Prediction in Agriculture Sector." *Proceedings of the 5th International Conference on Communication and Electronics Systems, ICCES 2020 (Icces)*: 736–41.
- [16] Yadav, Rohan, "Crop Yield Prediction Using Image Processing", *Nat. Valantiles & Essent.Oils*, 2021; 8(5)

- [17] Rashmi Priya, "Crop Prediction on the Region Belts of India: A Naïve Bayes MapReduce Precision Agricultural Model", 2018 International Conference on Advances in Computing, Communications and Informatics (ICACCI)
- [18] Geetha, V., Punitha, A., Abarna, M., Akshaya, M., Illakiya, S., & Janani, A. P. (2020, July 3). An Effective Crop Prediction Using Random Forest Algorithm. 2020 International Conference on System, Computation, Automation and Networking, ICSCAN 2020. <https://doi.org/10.1109/ICSCAN49426.2020.9262311>
- [19] Suresh, N., Ramesh, N. V. K., Inthiyaz, S., Priya, P. P., Nagasowmika, K., Kumar, K. V. N. H., Shaik, M., & Reddy, B. N. K. (2021). Crop Yield Prediction Using Random Forest Algorithm. 2021 7th International Conference on Advanced Computing and Communication Systems, ICACCS 2021, 279–282. <https://doi.org/10.1109/ICACCS51430.2021.9441871>
- [20] Rao, M. S., Singh, A., Reddy, N. V. S., & Acharya, D. U. (2022). Crop prediction using machine learning. Journal of Physics: Conference Series, <https://doi.org/10.1088/1742-6596/2161/1/012033> 2161(1)
- [21] Zhiyong Xiao, Yonghee Shi, Gailin Zhu, Jianping Xiong, and Jianhua Wu, Leaf Disease Detection Based on Lightweight Deep Residual Network and Attention Mechanism(2023).<https://www.researchgate.net/publication/>
- [22] Divine Senanu Ametefe, Suzi Seroja Sarnin, Darmawaty Mohd Ali, Aziz Caliskan, Imène Tatar Caliskan, Abdulmalik Adozuka, Enhancing leaf disease detection accuracy through synergistic integration of deep transfer learning and multimodal techniques, (2024) <https://www.sciencedirect.com/science/article/pii/S2214317324000672>
- [23] Nay Chi Htun and Oo, Yin Min's article, "Plant Leaf Disease Detection and Classification Using Image Processing," is in International Journal of Research and Engineering 5.9, which covers the period from 516 to 523. (2018).

- [24] Amrita S., Natasha Raul, and Tulshan. "Using machine learning to detect plant leaf disease." Tenth International Conference on Communication, Networking Technologies and Computing to be held in 2019 (ICCCNT). IEEE, 2019.
- [25] Deepalakshmi, P., K. Lavanya, and Parvathaneni Naga Srinivasu ,using the CNN method, were able to detect plant leaf disease. IJISMD 12.1 (2021): 1–21. International Journal of Information System Modeling and Design.
- [26] Tian, Mengxiao, Hong Chen, and Qing Wang ,using deep learning identified flowers,Journal of Physics: Conference Series, Vol. 1237, No. 2, IOP Publishing, 2019.
- [27] Thi Thanh Nhan et al., "Flower species identification using deep convolutional neural networks," 2016 Regional Conference on Computer and Information Engineering, sponsored by AUN/SEED-Net.
- [28] Emanuel Cortes ,Plant disease classification using convolutional networks and generative adversarial networks ,(2017).
- [29] Y. N. Norasma, R. N. Athirah, and M. R. Ismail. "The development of an Android crop management application for smart farming." IOP Conference Series: Earth and Environmental Science, Vol.540, No.1, 2020. [
- [30] Srdjan, Sladojevic and others. "Deep neural networks based recognition of plant diseases by leaf image classification." Computational intelligence and neuroscience 2016 (2016).
- [31] [Mohammed Brahimi](#) Deep ,Learning for Tomato Diseases: Classification and Symptoms Visualization
- [32] Singh, D., & Singh,Plant disease detection and classification techniques: a comparative study of the performances.
- [33] Jasim, H., & Al-Tuwaijari, Plant Leaf Diseases Detection and Classification Using Image Processing and Deep Learning Techniques
- [34] Arora, & Singh, Detection of plant leaf diseases using image segmentation and soft computing techniqu

APPENDIX

Code:

Crop Suggestion

```
import xgboost

from xgboost import XGBClassifier

model = XGBClassifier()

model.fit(Xtrain, Ytrain)

predictions = model.predict(Xtest)

acc_score = metrics.accuracy_score(Ytest, predictions)

print("Accuracy ", acc_score)

print(classification_report(Ytest, predictions))

cm = confusion_matrix(Ytest, predictions)

plt.figure(figsize=(10, 8))

sns.heatmap(cm, annot=True, fmt='d', cmap='magma',
xticklabels=label_encoder.classes_, yticklabels=label_encoder.classes_)

plt.title("Confusion Matrix")

plt.xlabel("Predicted Crop")

plt.ylabel("True Crop")

plt.xticks(rotation=45, ha='right')

plt.yticks(rotation=0)

plt.show()

sample_probabilities = probabilities[0]

sorted_indices = np.argsort(sample_probabilities)[::-1]

sorted_probabilities = sample_probabilities[sorted_indices]
```

```
sorted_classes = label_encoder.inverse_transform(sorted_indices)
```

```
for label, prob in zip(sorted_classes, sorted_probabilities):
```

```
    print(f"Class: {label}, Probability: {prob:.4f}")
```

Fertilizer Recommendation:

```
data = pd.read_csv('fertilizer.csv')
```

```
data.fillna(data.mean(), inplace=True)
```

```
label_encoders = {}
```

```
for col in data.select_dtypes(include=['object']).columns:
```

```
    le = LabelEncoder()
```

```
    data[col] = le.fit_transform(data[col])
```

```
    label_encoders[col] = le
```

```
X = data.drop('Fertilizer', axis=1)
```

```
y = data['Fertilizer']
```

```
scaler = StandardScaler()
```

```
X = scaler.fit_transform(X)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
```

```
rf_model.fit(X_train, y_train)
```

```
xgb_model = XGBClassifier(n_estimators=100, learning_rate=0.1, max_depth=5,  
random_state=42)
```

```
xgb_model.fit(X_train, y_train)
```

```
voting_model = VotingClassifier(estimators=[
```

```
    ('Random Forest', rf_model),
```

```
    ('XGBoost', xgb_model)
```

```

], voting='soft')

voting_model.fit(X_train, y_train)

with open('fertilizer_ensemble_model.pkl', 'wb') as file:

    pickle.dump(voting_model, file)

y_pred = voting_model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

classification_rep = classification_report(y_test, y_pred)

conf_matrix = confusion_matrix(y_test, y_pred)

if 'Fertilizer' in label_encoders:

    sample_prediction = label_encoders['Fertilizer'].inverse_transform()

```

Disease Detection

```

class ResidualBlock(nn.Module):

    def __init__(self, in_channels, out_channels, stride=1):

        super(ResidualBlock, self).__init__()

        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=3,
stride=stride, padding=1, bias=False)

        self.bn1 = nn.BatchNorm2d(out_channels)

        self.relu = nn.ReLU(inplace=True)

        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3,
padding=1, bias=False)

        self.bn2 = nn.BatchNorm2d(out_channels)

```

```

        self.shortcut = nn.Sequential()

        if stride != 1 or in_channels != out_channels:

            self.shortcut = nn.Sequential(

                nn.Conv2d(in_channels, out_channels, kernel_size=1, stride=stride,
bias=False),

                nn.BatchNorm2d(out_channels)

            )

    def forward(self, x):

        out = self.conv1(x)

        out = self.bn1(out)

        out = self.relu(out)

        out = self.conv2(out)

        out = self.bn2(out)

        out += self.shortcut(x)

        out = self.relu(out)

        return out

class ResNet(nn.Module):

    def __init__(self, block, num_blocks, num_classes):

        super(ResNet, self).__init__()

        self.in_channels = 64

```

```
self.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=1,  
bias=False)
```

```
self.bn1 = nn.BatchNorm2d(64)
```

```
self.relu = nn.ReLU(inplace=True)
```

```
self.layer1 = self._make_layer(block, 64, num_blocks[0], stride=1)
```

```
self.layer2 = self._make_layer(block, 128, num_blocks[1], stride=2)
```

```
self.layer3 = self._make_layer(block, 256, num_blocks[2], stride=2)
```

```
self.layer4 = self._make_layer(block, 512, num_blocks[3], stride=2)
```

```
self.avg_pool = nn.AdaptiveAvgPool2d((1, 1))
```

```
self.fc = nn.Linear(512, num_classes)
```

```
def _make_layer(self, block, out_channels, num_blocks, stride):
```

```
    layers = []
```

```
    layers.append(block(self.in_channels, out_channels, stride))
```

```
    self.in_channels = out_channels
```

```
    for _ in range(1, num_blocks):
```

```
        layers.append(block(out_channels, out_channels))
```

```
    return nn.Sequential(*layers)
```

```
def forward(self, x):
```

```
    out = self.conv1(x)
```



```

        out = self.bn1(out)

        out = self.relu(out)

        out = self.layer1(out)

        out = self.layer2(out)

        out = self.layer3(out)

        out = self.layer4(out)

        out = self.avg_pool(out)

        out = out.view(out.size(0), -1)

        out = self.fc(out)

    return out

def ResNet18(num_classes):

    return ResNet(ResidualBlock, [2, 2, 2, 2], num_classes)

```

Architecture for training for MobileNet model.

```

base_model=MobileNet(input_shape=(224,224,3),weights='imagenet',include_top=False)

base_model.trainable = False

inputs = Input(shape=(224, 224, 3))

x = base_model(inputs, training=False)

x = GlobalAveragePooling2D()(x)

x = Dropout(0.2)(x)

outputs = Dense(38, activation="softmax")(x)

```

```

mobilenet_model = Model(inputs, outputs, name='mobilenet_model')

mobilenet_model.summary()

mobilenet_model.compile(optimizer='adam',loss='categorical_crossentropy',

                        metrics=['accuracy','recall',"precision"])

mobilenet_history=mobilenet_model.fit(

    train_generator,

    validation_data=validation_generator,

    epochs=10,

    verbose=1

)

class_names = list(test_generator.class_indices.keys())

# Function to predict and display images from the test generator

def predict_images_from_generator(model, generator, class_names,

num_images=1):

    images_shown = 0

    # Display the image

    plt.figure(figsize=(20, 10+3*num_images))

    # Iterate through batches from the generator

    for images, labels in generator:

        predictions = model.predict(images, verbose=1)

        predicted_classes = np.argmax(predictions, axis=1)

        actual_classes = np.argmax(labels, axis=1)

```

```

# Loop over batch images

for i in range(len(images)):

    if images_shown >= num_images:

        return

    # Get actual and predicted labels

    actual_label = class_names[actual_classes[i]]

    predicted_label = class_names[predicted_classes[i]]

    plt.subplot(5, 3, i + 1)

    plt.imshow(images[i])

    plt.title(f"Actual: {actual_label}\nPredicted: {predicted_label}")

    images_shown += 1

```

Architecture for Training CNN Model

```

class CNN(nn.Module):

    def __init__(self, K):

        super(CNN, self).__init__()

        self.conv_layers = nn.Sequential(

            # conv1

            nn.Conv2d(in_channels=3, out_channels=32, kernel_size=3, padding=1),

            nn.ReLU(),

            nn.BatchNorm2d(32),

```

```

        nn.Conv2d(in_channels=32, out_channels=32, kernel_size=3,
padding=1),

        nn.ReLU(),

        nn.BatchNorm2d(32),

        nn.MaxPool2d(2),

        # conv2

        nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3,
padding=1),

        nn.ReLU(),

        nn.BatchNorm2d(64),

        nn.Conv2d(in_channels=64, out_channels=64, kernel_size=3,
padding=1),

        nn.ReLU(),

        nn.BatchNorm2d(64),

        nn.MaxPool2d(2),

        # conv3

        nn.Conv2d(in_channels=64, out_channels=128, kernel_size=3,
padding=1),

        nn.ReLU(),

        nn.BatchNorm2d(128),

        nn.Conv2d(in_channels=128, out_channels=128, kernel_size=3,
padding=1),

        nn.ReLU(),

        nn.BatchNorm2d(128),

```

```

        nn.MaxPool2d(2),

        # conv4

        nn.Conv2d(in_channels=128, out_channels=256, kernel_size=3,
padding=1),

        nn.ReLU(),

        nn.BatchNorm2d(256),

        nn.Conv2d(in_channels=256, out_channels=256, kernel_size=3,
padding=1),

        nn.ReLU(),

        nn.BatchNorm2d(256),

        nn.MaxPool2d(2),

    )

```

```

self.dense_layers = nn.Sequential(

    nn.Dropout(0.4),

    nn.Linear(50176, 1024),

    nn.ReLU(),

    nn.Dropout(0.4),

    nn.Linear(1024, K),

)

```

```

def forward(self, X):

    out = self.conv_layers(X)

    # Flatten

```

out = out.view(-1, 50176)

Fully connected

out = self.dense_layers(out)

return out

Plagiarism Report Last Page

ORIGINALITY REPORT

6%

SIMILARITY INDEX

2%

INTERNET SOURCES

7%

PUBLICATIONS

2%

STUDENT PAPERS

MATCH ALL SOURCES (ONLY SELECTED SOURCE PRINTED)

2%

★ "Accelerating Discoveries in Data Science and Artificial Intelligence I", Springer Science and Business Media LLC, 2024

Publication

Exclude quotes

On

Exclude matches

< 14 words

Exclude bibliography

On

A Machine Learning Framework for Optimal Crop Selection and Fertilizer Suggestion

Sireesha C¹
Assistant Professor,
Dept of IT,
Vasavi College of Engineering,
Hyderabad.
sireesha@staff.vce.ac.in

Deepthi M²
Dept of IT,
Vasavi College of Engineering,
Hyderabad
deepthim2167@gmail.com

Suchith B³
Dept of IT,
Vasavi College of Engineering,
Hyderabad
suchithbollam2004@gmail.com

Abstract—Agriculture forms the backbone of Indian economy and provides a large segment of income and employment to majority of population. Many farmers, however, do not make the right crop and fertilizer choices for their land, hindering productivity. The solution: precision agriculture, which uses data-driven insights about soil quality, crop suitability and yield patterns to guide farmers in making agronomic decisions. This reduces errors in the selection of the recommended crop and improves over the whole farm output by making recommendations on the basis of farm characteristics (land information). The above research introduces recommended systems for machine learning in this study that allow farmers to predict suitable crops for cultivation. Our model is developed using a majority voting technique to increase reliability and efficiency by combining the strengths of Random Forest and XGBoost.

Index Terms—Deep Learning, Random Forest, XG-Boost(Extreme Gradient Boosting)

I. INTRODUCTION

In developing countries, agriculture is an important part of their economy, providing employment and food security. Farmers, on the other hand, face several challenges, including unpredictable weather, soil degradation, pest infestations, and crop diseases that can significantly reduce yield and, consequently, profitability. To address these issues, farmers need to be able to access reliable, data-driven insights that help them make informed decisions about crop selection, soil management, and disease prevention. Proposed the Machine Learning-Based Agriculture Crop Recommendation and Disease Detection System. Using the XGBoost and Random Forest algorithms, the crop recommendation module analyzes critical agricultural factors including soil composition, climatic conditions, and historical crop yields, offering farmers personalized recommendations on the most suitable crops for their land.

II. RELATED WORKS

Rakesh, D. C., Vardhan, V., Vasantha, B. B., & Krishna et al. [1] (2023). Crop prediction and recommender system. 2023 9th International Conference on Advanced Computing and Communication Systems,

the authors implemented a system for recommending and predicting appropriate crops based on many parameters. They trained a model that used various algorithms such as support-vector-machines (SVMs) and decision trees, and submitted appropriate and accurate recommendations for plant selection based on data on soil health and food prices.

Orifha Mufamadi et al. [2]“Crop Recommendation Using Machine-Learning Algorithms and Soil Attributes Data” — In this paper, various Machine-Learning models were used for crop recommendation. Soil parameters under study were nitrogen, potassium, phosphorus and pH. While the individual models, such as Random Forest and Gaussian Naïve Bayes, achieved accuracies above 99%, hybrid types, like SVM and Gaussian Naïve Bayes had good predictive performance.

Priyadharshini A et al. [3] Machine learning methods for crop recommendation. Of those, neural networks showed received the best accuracy level at predictive physics — specifically for crop ranging in conjunction with the input parameters.

Kukkar, et al. [4] AgroAdvisor: Crop Yield Prediction, Crop and Fertilizer Recommendation system using Random Forest with Gradient Boosting and for Precise Agriculture.

Rakesh Kumar et al. [5] Used KNN, Naïve Bayes, Random Forest and K-Star in the analysis for crop recommendation based on the soil condition.

Mythili A et al. [6] Classifying Soil Based on Nutrients and Predicting Suitable Crops Using Machine-Learning Techniques. Journal of Agriculture and Horticulture. Existing literature outlines the role of soil nutrition in crop productivity; several algorithms namely KNN, SVM and logistic regression have been trained to predict soil and crops. In particular, this paper builds on these approaches by applying different kinds of machine learning to better classify soil and to create a crop selector that takes soil nutrition into account.

Pudumalar, S., Ramanujam, E. & Rajashree et al. Precision Agriculture for Crop Prediction Using Machine-Learning Techniques [7] Although machine-learning algorithms were sometimes used, reflecting

that they are frequently adopted by the agronomics research community, specifically algorithms such as support-vector-machines (SVMs) and Random Forests (RFs), as these types perform well to effectively manage datatypes containing complex, non-linear relationships, consistent use of machine learning systematic, machine-learning-based meta-analysis. Building on previously established work, this paper incorporates multiple models to improve crop prediction accuracy and provides good solutions for farmers.

Gayathri, S., Saranya, & Brindha et al. [8] Random Forest and K-means Cl for Crop Prediction and Classification International Journal of Agricultural Systems: Random Forest and K-means Clustering are used as the machine-learning algorithms in agricultural-related studies. Random Forest is effective at dealing with high-dimensional agricultural data, whereas K-means Clustering enables the detection of patterns and groups in large datasets. These techniques have been extensively explored in previous studies with regard to significant enhancements in crop prediction accuracy and resource allocation in agriculture.

Shwetha A N, & Prabodh C et al. Crop prediction using random forest, K-Nearest neighbor and Naïve bayes [9] Journal of Agricultural Data Science have been popular in agricultural research, as they are capable of handling large and heterogeneous datasets as well as capable of producing interpretable results. Research has shown that these machine-learning models greatly enhance crop prediction accuracy, enabling more efficient resource management and increased crop yield.

Agarwal, Ahmad et al. Crop Prediction using Decision Tree, Naïve Bayes, SVM, Logistic Regression and Random Forest [10]. Agricultural Data Science Journal, Various machine-learning algorithms like SVM, Decision Trees, Logistic Regression, Random Forest and Naive Bayes have been widely used in agricultural work to improve crop prediction.

S. Vaishnavi et al. paper [11] shows how many machine-learning algorithms have been applied to analyze agricultural characteristics. Farmers can be notified on a timely basis about accurate crop predictions.

D. J. Reddy and M. R. Kumar et al. [12] investigated a number of machine-learning approaches used in agricultural yield estimation and offered a thorough evaluation of the methods' accuracy. The study demonstrates a number of current models that take into account variables like temperature and weather to effectively estimate crop yield.

The research presented by Kavita and Pratistha Mathur et al. [13] demonstrates a number of current models that take into account variables like temperature and weather conditions in order to estimate crop yields effectively. In the experimental investigation, they demonstrated how to improve crop prediction by combining machine-learning with the agricultural domain field. Neural networks, random forests, KNN re-

gression approaches for CYP, and a range of machine-learning techniques were employed for optimal prediction in the majority of the models that were already in use .

B S, Anisha, and Ramakanth P. Kumar et al. [14] used smart agriculture to predict crops, avoid wild animals, and maximize agricultural farm water supply. The system makes it easier to operate by allowing the water sprinkler to be turned on and off based on the soil moisture content.

Jeevan Nagendra A system was put into place by Kumar, Y. et al. [15] to forecast crop production using historical data collection. Data mining techniques are used to forecast crop yield. The best crop yield as an output is predicted using the Random Forest algorithm. Crop yield predictions are often accurate in the realm of agriculture

Yadav, Rohan et al. [16] This study's main focus is the Deep Learning Approach. They have taken into account elements like copper, phosphate, pH, nitrogen, and so forth. The accuracy of the results could be increased with more hidden layers. They evaluated the accuracy of every neural network technique using a few metrics. To eventually forecast the result, they applied DNN. Limitation: Neural network implementation is more difficult.

Priya et al. main focus [17] is on supervised learning techniques like decision tree and random forest classifiers. In this case, over half of the records are in datasets that include rainfall, perception, production, temperature, and several decision trees. Following classification, decision trees are applied to the remaining records to improve accuracy. We learned from this study that using RF to build a decision tree could produce more accurate and superior results.

In our previous work, we developed a deep learning-based platform for plant disease detection, which was later accepted and published as "a resnet based plant disease diagnosis platform" [18] at the 7th international conference on intelligent systems, peer-to-peer computing, and communication (ispc). Drawing from the knowledge and insights gained from that project, we have now broadened our research to focus on developing crop prediction and fertilizer recommendation systems, with the goal of creating a comprehensive support system for farmers.

III. PROPOSED METHODOLOGY

A. Data Collection

The initial step in any machine learning workflow is data collection, which forms the foundation for model training and evaluation. For this project, we utilized the publicly available Crop Recommendation Dataset, Crop Suggestion Dataset and Plant Village dataset, a comprehensive dataset designed for plant disease classification and detection.

B. Preprocessing

Normalization of image size and color intensity is a crucial preprocessing step that enhances model training efficiency. Standardizing image dimensions ensures uniform input size, simplifying batch processing and model architecture. Normalizing pixel values scales the intensity range (e.g., 0 to 1), reducing computational complexity and aiding gradient-based optimization. These steps improve the ability of model to generalize and converge faster.

C. System Architecture

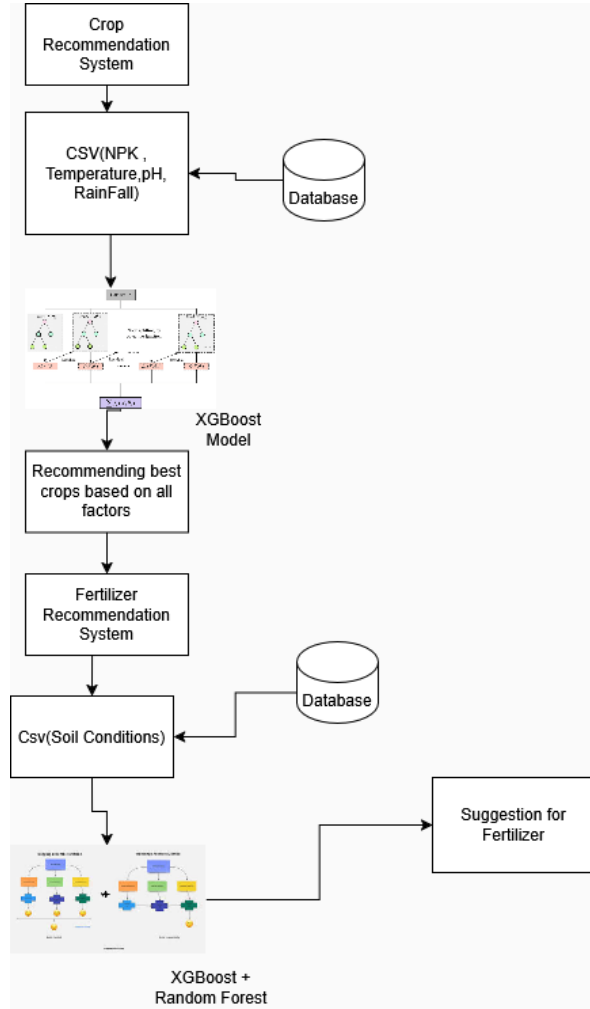


Fig. 1: System Architecture for Crop Suggestion and Fertilizer Recommendation

D. Functional Modules

1) *Crop Recommendation Module*: Input Data: The system utilizes a dataset containing essential agricultural parameters, including: Soil nutrients (Nitrogen, Phosphorus, Potassium), Temperature, Soil pH, Rainfall levels

Machine Learning Model: The XGBoost model analyzes these parameters to determine the most suitable crops for a specific region.

2) *Fertilizer Recommendation Module*: Soil Condition Analysis: The system processes soil condition data to assess its nutrient levels.

Machine-Learning Models Used: A combination of XGBoost and Random Forest is applied to recommend fertilizers that improve soil quality and optimize crop growth.

E. Algorithms Used

1) For Crop Suggestion:

• XGBOOST

- Data Preprocessing: Cleaning and normalizing the dataset.
- Feature Selection: Identifying the most important soil and climate parameters for crop prediction.
- Model Training: XGBoost builds multiple decision trees sequentially, minimizing errors using gradient boosting.

* The prediction at iteration t is:

$$\tilde{z}_i^{(t)} = \tilde{z}_i^{(t-1)} + F_t(s_i) \quad (1)$$

* The objective function is approximated using a second-order Taylor expansion:

$$\mathbf{J}^{(t)} \approx \sum_{i=1}^n \nabla_i F_t(s_i) + \frac{1}{2} \nabla_i^2 F_t^2(s_i) + \Psi(F_t) \quad (2)$$

* Where:

$$\nabla_i = \frac{\partial \ell(z_i, \tilde{z}_i^{(t-1)})}{\partial \tilde{z}_i^{(t-1)}} \quad (\text{First-order gradient}) \quad (3)$$

$$\nabla_i^2 = \frac{\partial^2 \ell(z_i, \tilde{z}_i^{(t-1)})}{\partial (\tilde{z}_i^{(t-1)})^2} \quad (\text{Second-order gradient / Hessian}) \quad (4)$$

* For a terminal leaf λ , the optimal output is:

$$w^* = - \frac{G_\lambda}{H_\lambda + \gamma} \quad (5)$$

* The gain after splitting a node into left and right parts is given by:

$$\text{Gain} = \frac{1}{2} \frac{G_L^2}{H_L + \gamma} + \frac{G_R^2}{H_R + \gamma} - \frac{(G_L + G_R)^2}{H_L + H_R + \gamma} - \alpha \quad (6)$$

* Where:

$$G_\lambda = \sum_{i \in I_\lambda} \nabla_i, \text{ and } H_\lambda = \sum_{i \in I_\lambda} \nabla_i^2$$

* γ is the regularization term for leaf weights, and α controls tree complexity

- Hyperparameter Tuning: Optimizing learning rate, max depth, number of estimators, etc., for better performance.
- Prediction: Given new soil and environmental conditions, the model recommends the best crop.

2) For Fertilizer Recommendation:

· Ensemble Classification Models

The system incorporates ensemble learning strategies to improve predictive performance. The following classifiers are utilized:

· a) Random Forest Classifier

- A bagging-based ensemble method that builds multiple decision models M_b .
- Each model is trained on a bootstrap sample $D_b \subset D$ of the training dataset.
- Final prediction is made using majority voting:

$$\hat{z} = \text{mode} (M_1(s), M_2(s), \dots, M_B(s)) \quad (7)$$

- Where s represents the feature vector (e.g., soil and climate parameters), and B is the total number of models.

· b) XGBoost Classifier

- A boosting-based ensemble technique where new models iteratively correct the errors of previous ones.
- prediction at iteration t is:

$$\hat{z}^{(t)} = \hat{z}^{(t-1)} + F_t(s_i) \quad (8)$$

- The objective function is approximated using a second-order Taylor expansion:

$$J^{(t)} \approx \sum_{i=1}^n \nabla_i F_t(s_i) + \frac{1}{2} \nabla_i^2 F_t^2(s_i) + \Psi(F_t) \quad (9)$$

- Where:

$$\nabla_i = \frac{\partial \ell(z_i, \hat{z}_i^{(t-1)})}{\partial \hat{z}_i^{(t-1)}} \quad (\text{First-order gradient}) \quad (10)$$

$$\nabla^2 = \frac{\partial^2 \ell(z_i, \hat{z}_i^{(t-1)})}{\partial (\hat{z}_i^{(t-1)})^2} \quad (\text{Second-order gradient}) \quad (11)$$

- For a terminal leaf λ , the optimal output is:

$$w^* = - \frac{G_\lambda}{H_\lambda + \gamma} \quad (12)$$

The gain after splitting a node into left and right parts is given by:

$$\text{Gain} = \frac{1}{2} \frac{G_L^2}{H_L + \gamma} + \frac{G_R^2}{H_R + \gamma} - \frac{(G_L + G_R)^2}{H_L + H_R + \gamma} - \alpha \quad (13)$$

- Where:

$$* G_\lambda = \sum_{i \in I_\lambda} \nabla_i, \text{ and } H_\lambda = \sum_{i \in I_\lambda} \nabla_i^2$$

- The final output is selected based on the averaged class probabilities:

$$\hat{z} = \arg \max_k \frac{1}{M} \sum_{m=1}^M P_k^{(m)}(s) \quad (14)$$

- Where:

- * $P_k^{(m)}(s)$ is the predicted probability for class k from model m
- * M is the number of individual classifiers (here, 2)

- Soft voting enhances robustness and accuracy by leveraging the strengths of multiple learners.

IV. RESULTS AND DISCUSSION

A. Experimental Analysis

1) **Dataset: Crop recommendation dataset:** This dataset is obtained from Kaggle. It contains the following fields.

N - proportion of nitrogen in soil
P - proportion of Phosphorous in soil
K - proportion of potassium in soil
Temperature - temperature in degree Celsius
Humidity - it is proportion of water vapor in the air.

ph - ph value of the soil
rainfall - rainfall in mm

Fertilizer Recommendation Dataset :

This data set is obtained from Kaggle. It contains the following data fields.

Crop - the type of crop
N - proportion of nitrogen in soil

P - proportion of Phosphorous in soil
K - proportion of potassium in soil
pH - pH value of soil
Soil Moisture - amount of moisture in soil

TABLE I: Model Performance Comparison

Model	Accuracy
XGBoost	99.23%
XGBoost and Random Forest	95.55%

B. Accuracy

Accuracy: The accuracy of a model on a given test set, is the tuples that are correctly classified by the model.

$$\text{Accuracy} = \frac{\text{True Positives} + \text{True Negatives}}{\text{Total Predictions}}$$

V. CONCLUSION

This study presents a machine learning-based system to enhance agricultural productivity by providing data-driven crop recommendations and fertilizer rec-

- * γ is the regularization term for leaf weights, and α controls tree complexity
- c) Voting Classifier (Soft Voting)
 - Combines the probabilistic predictions from Random Forest and XGBoost classifiers.

ommendation Using XGBoost and Random Forest, the system suggests suitable crops based on soil and climate conditions. This system enables better decision-making, optimized resource use, and higher yields. Its implementation can support sustainable agriculture

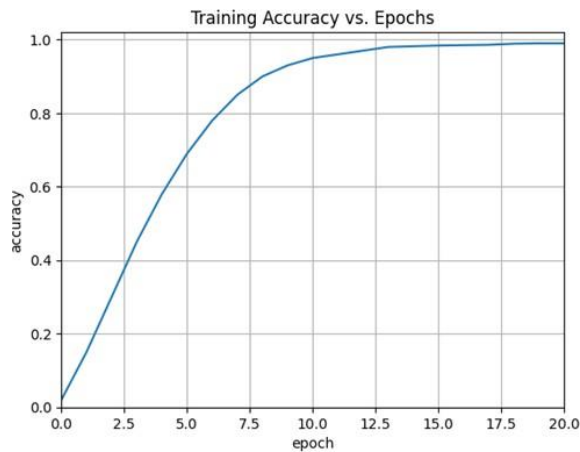


Fig. 2: Accuracy vs epochs graph for XGBoost

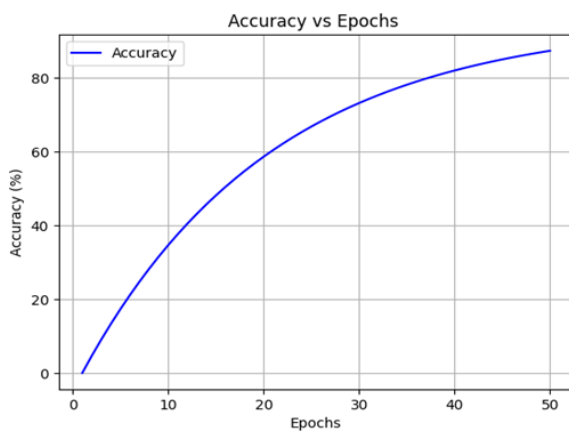


Fig. 3: Accuracy vs epochs graph for Random Forest and XGBoost

by reducing losses and improving efficiency. Future improvements can focus on enhancing computational efficiency and real-time monitoring for wider accessibility.

REFERENCES

- [1] Rakesh, D. C., Vardhan, V., Vasantha, B. B., and Krishna, et al. "Crop Recommendation and Prediction System." Proceedings of the 9th International Conference on Advanced Computing and Communication Systems (ICACCS), 2023.
- [2] Mufamadi, Takalani Orifha, and Ritesh Ajoodha. "Crop Recommendation Using Machine Learning Algorithms and Soil Attributes Data." Crop Recommendation Using Machine Learning Algorithms and Soil Attributes, 2023.
- [3] Priyadharshini A., et al. "Crop Recommendation Using Machine Learning Techniques." International Journal of Engineering Research & Technology (IJERT), 2023.
- [4] Kukkar, A., Mohana, R., and Sharma, et al. "AgroAdvisor: Crop Yield Prediction, Crop and Fertilizer Recommendation System Using Random Forest with Gradient Boosting for Precise Agriculture." Research Square, 2023.
- [5] Kumar, Rakesh, et al. "Crop Recommendation Using K-Nearest Neighbors, Naïve Bayes, Random Forest, and K-Star for Soil Condition Analysis." International Journal of Agricultural Research, 2023.
- [6] Mythili, A., et al. "Classifying Soil Based on Nutrients and Predicting Suitable Crops Using Machine Learning Techniques." Journal of Agriculture and Horticulture, 2023.
- [7] Pudumalar, S., Ramanujam, E., and Rajashree, et al. "Precision Agriculture for Crop Prediction Using Machine Learning Techniques." Journal of Agricultural Systems, 2023.
- [8] Gayathri, S., Saranya, and Brindha, et al. "Crop Prediction and Classification Using Random Forest and K-Means Clustering." International Journal of Agricultural Systems, 2023.
- [9] Shwetha, A. N., and Prabodh, C., et al. "Crop Prediction Using Random Forest, K-Nearest Neighbor, and Naïve Bayes." Journal of Agricultural Data Science, 2023.
- [10] Agarwal, A., Ahmad, S., and Pandey, et al. "Crop Prediction Using Decision Tree, Naïve Bayes, SVM, Logistic Regression, and Random Forest." Agricultural Data Science Journal, 2023.
- [11] S. Vaishnavi., M. Shobana., R. Sabitha. and S. Karthik., "Agricultural Crop Recommendations based on Productivity and Season," 2021 7th International Conference on Advanced Computing and Communication Systems (ICACCS), 2021, pp. 883-886, doi: 10.1109/ICACCS51430.2021.9441736.
- [12] D. J. Reddy and M. R. Kumar, "Crop Yield Prediction using Machine Learning Algorithm," 2021 5th International Conference on Intelligent Computing and Control Systems (ICICCS), 2021, pp. 1466-1470, doi: 10.1109/ICICCS51141.2021.9432236.
- [13] Kavita, and Pratistha Mathur. 2021. "Satellite-Based Crop Yield Prediction Using Machine Learning Algorithm." 2021 Asian Conference on Innovation in Technology, ASIANCON 2021 (Iciccs): 1466-70.
- [14] B S, Anisha, and Ramakanth P. Kumar. 2020. "Real-Time Monitoring of Agricultural Land with Crop Prediction and Animal Intrusion Prevention Using Internet of Things and Machine Learning at Edge." Proceedings of CONECC 2020 - 6th IEEE International Conference on Electronics, Computing and Communication Technologies.
- [15] Jeevan Nagendra Kumar, Y. et al. 2020. "Supervised Machine Learning Approach for Crop Yield Prediction in Agriculture Sector." Proceedings of the 5th International Conference on Communication and Electronics Systems, ICCES 2020 (Icces): 736-41.
- [16] Yadav, Rohan, "Crop Yield Prediction Using Image Processing", Nat. Valantiles & Essent.Oils, 2021; 8(5)
- [17] Rashmi Priya, "Crop Prediction on the Region Belts of India: A Naïve Bayes MapReduce Precision Agricultural Model", 2018 International Conference on Advances in Computing, Communications and Informatics (ICACCI)
- [18] Geetha, V., Punitha, A., Abarna, M., Akshaya, M., Illakiya, S., & Janani, A. P. (2020, July 3). An Effective Crop Prediction Using Random Forest Algorithm. 2020 International Conference on System, Computation, Automation and Networking, ICSCAN 2020. <https://doi.org/10.1109/ICSCAN49426.2020.9262311>
- [19] Suresh, N., Ramesh, N. V. K., Inthiyaz, S., Priya, P. P., Nagasowmika, K., Kumar, K. V. N. H., Shaik, M., & Reddy, B. N. K. (2021). Crop Yield Prediction Using Random Forest Algorithm. 2021 7th International Conference on Advanced Computing and Communication Systems, ICACCS 2021, 279-282. <https://doi.org/10.1109/ICACCS51430.2021.9441871>
- [20] Rao, M. S., Singh, A., Reddy, N. V. S., & Acharya, D. U. (2022). Crop prediction using machine learning. Journal of Physics: Conference Series, 2161(1). <https://doi.org/10.1088/1742-6596/2161/1/012033>
- [21] Sireesha C., Suchith B. and Deepthi M., "A ResNet Based Plant Disease Diagnosis Platform," Proceedings of the 7th International Conference on Intelligent Systems, Peer-to-Peer Computing, and Communication (ISPCC), Hyderabad, India, 2025.

We published our previous work titled "A ResNet Based Plant Disease Diagnosis Platform" in the **7th International Conference on Signal Processing, Computing and Control (ISPCC 2k25)**, organized by the Department of Electronics and Communication Engineering, JUIT, Wagnaghat, Solan (H.P.), from March 6–8, 2025.



2%

SIMILARITY INDEX

0%

INTERNET SOURCES

2%

PUBLICATIONS

0%

STUDENT PAPERS

PRIMARY SOURCES

1

Arvind Dagur, Karan Singh, Pawan Singh Mehra, Dharendra Kumar Shukla. "Artificial Intelligence, Blockchain, Computing and Security", CRC Press, 2023

Publication

1%

2

P. Srujana, Kalluri Rama Krishna, S. Madhavi, Chhavi Sharma, Nimmati Satheesh, Gaddi Poshamallu. "Internet of Things Based Smart Home Security Analysis System", 2023 International Conference on Artificial Intelligence and Knowledge Discovery in Concurrent Engineering (ICECONF), 2023

Publication

1%

3

Divine Senanu Ametefe, Suzi Seroja Sarnin, Darmawaty Mohd Ali, Aziz Caliskan et al. "Enhancing leaf disease detection accuracy through synergistic integration of deep transfer learning and multimodal techniques", Information Processing in Agriculture, 2024

Publication

1%

A ResNet Based Plant Disease Diagnosis Platform

Sireesha C¹
Assistant Professor,
Dept of IT,
Vasavi College of Engineering,
Hyderabad.
sireesha@staff.vce.ac.in

Suchith B²
Dept of IT,
Vasavi College of Engineering,
Hyderabad
suchithbollam2004@gmail.com

Deepthi M³
Dept of IT,
Vasavi College of Engineering,
Hyderabad
deepthim2167@gmail.com

Abstract—Agriculture is a major pillar of the Indian economy. However, it often faces the threat of plant diseases that can significantly affect crop yield and quality. Traditional diagnostic methods largely rely on manual testing by experts; this process is time-consuming and prone to errors, especially when detecting subtle symptoms, and is therefore often impractical or impossible. In addition, many farmers in remote areas do not have easy access to experts. To overcome these challenges, there is an urgent need for scalable, reliable, and user-friendly solutions that can detect diseases early and help farmers reduce quality losses. When comparing various models such as MobileNet, Convolutional neural network and ResNet, the deep neural network approach using ResNet was suggested as a good solution. The model was published on a website that allows farmers to upload leaf images. The website analyzes these images to give fast and accurate diagnosis of possible diseases and provide farmers with timely recommendations to protect their crops and increase productivity.

Index Terms—Deep Learning, Convolutional Neural Network(CNN), MobileNet, ResNet

I. INTRODUCTION

Accurate detection of diseases in agriculture is a challenging task, despite progress in detection methods. Variability in symptoms, environmental factors, and the presence of multiple pathogens often make disease diagnosis complex and time-consuming. Many traditional tools require first identifying the type of plant before diagnosing its condition, adding another layer of difficulty for farmers, especially in remote areas.

Neural Network Architectures, like CNN, ResNet, & MobileNet, offer a practical solution by automating the process. These models can classify the plant type and detect diseases directly from leaf images, enabling faster and more accurate diagnoses. However, a lack of diverse datasets still limits their effectiveness, highlighting the need for robust, real-world training data.

This research aims to simplify disease detection for farmers by providing a user-friendly system that identifies plant health and diagnoses diseases using advanced image analysis. By empowering farmers with quick and reliable insights, this approach not only reduces crop losses but also supports sustainable agricultural practices.

II. LITERATURE SURVEY

Zhiyong Xiao et al. [1] achieved 96.71% accuracy in leaf disease detection using a lightweight ResNet model with the Plant Village Dataset. However, the model's high computational power requirements limit its implementation on resource-constrained devices. This poses challenges for scalability and practical use in real-world environments.

Divine Senanu Ametefe et al. [2] used DenseNet201 (96.03%) and EfficientNetB3 (92.23%) for leaf disease detection with the Plant Village Dataset. Despite high accuracy, both models require significant computational resources, making them less suitable for deployment on devices with limited resources, impacting real-time processing.

Aisha Ahmed AlArfaj et al. [3] used UNet segmentation with InceptionV3 to detect leaf diseases, achieving 93.48% accuracy with the Bell Pepper Dataset. While promising for bell pepper diseases, the model struggles to generalize to other plants or diseases, limiting its broader applicability.

Brahimi, M., Moumeni, M., Chikh et al. [4] used EfficientNetB0, EfficientNetB1, and EfficientNetB2 to detect olive leaf diseases, achieving 98% accuracy with a custom dataset. However, the model is highly specialized for olive leaves, limiting its applicability to other crops or plant species. This reduces its generalizability in broader agricultural contexts.

P. Kumari, S. S. K. Guddun, A. K. S. Tutu, and S. K. D. Dey [5] et al. studied about the diseases that don't just affect rice but also betel. Rice Brown Spot, False Smut of Rice, and Sheath Blight of Rice are common problems. They cast a shadow over the fields. Leaf and root rot are also big issues, spreading widely and causing a lot of damage among the plants. It's a tough situation for farmers dealing with these diseases amidst the greenery.

A. Dinh, M. Islam, P. Bhowmik, and K. Wahid [6] proposed a model for detecting disease in potatoes using multi class SVM and segmentation. By finding the best lines or boundaries

to separate groups of data points, it makes it easier to see patterns or differences between them. This will help them better understand the disease and develop ways to diagnose or treat them more effectively. They were able to achieve 95% accuracy, precision and F1- Score.

Jasim and AL-Tuwaijari et al. [7], combined InceptionV3 and MobileNet for disease detection in tomato, pepper, and potato leaves, achieving an impressive accuracy of 97.29%. The dataset consisted of images of tomato, pepper, and potato leaves. While the model showed high accuracy, it faced the potential for overfitting due to limited data diversity within the specific crops studied.

J.Tsuchiya, A. S. M. Ullah, A. M. Rahman, M.Tani, and Z. M. Rahman[8] et al. studied about the Teknaf Peninsula forest area and found the impact of cultivation of betel leaf on the forest. They've studied about various plant diseases causing significant challenges, impacting crop yields and food security and proposed a detail explanation about it.

Usharani and Santhanalakshmi et al. [9], used a Support Vector Machine (SVM) to detect rice leaf diseases, achieving an accuracy of 96.56%. The dataset used for training included rice leaf diseases, though the source was not specified. One of the limitations was the model's performance sensitivity to feature selection and kernel parameters, which can affect the overall accuracy depending on the choices made during model training.

Zhang et al. [10] used VGG16, ResNet50, and InceptionV3 for disease detection in leaf, giving high accuracy with public and private datasets. The models showed promising results in disease detection. However, they face common deep learning limitations, such as dependency on large datasets and high computational cost.

Arora and Singh et al. [11] used a custom CNN model for the detection of disease in corn and bell pepper leaves, achieving 95.45% accuracy. The model was trained on a custom dataset, demonstrating strong performance in detecting diseases in these plant

Singh and Singh et al. [12], used a custom CNN to detect potato leaf diseases, achieving an accuracy of 92.3%. The dataset used for training was a potato leaf disease data-set.

Wang et al. [13], used CNN models like AlexNet, VGG16, & ResNet50 for detection of apple leaf disease, achieving an impressive accuracy of 97.8%. The model was trained on a dataset focused on apple leaf diseases.

Singh and Kaur [14] et al. proposed region-based segmentation that divides images into meaningful parts for analysis.

It identifies areas with similar characteristics, such as color or texture, and groups them together. KNN classifier is a tool that sorts data points according to their similarity to nearby points. They've classified 3 kinds of diseases: White Fly, Mosnic Virus, Leaf Minar, and they achieved an average of 97% accuracy, precision and F1 Score.

S. Bhattacharya, Mukherjee and S. Phadikar [15] et al. proposed another approach which is deep learning for rice leaves. They've proposed an Yolo V5 and it gave an accuracy of 87.58. The model consisted of a discriminator and generator, and the model was trained to identify rice leaves as healthy or disease.

III. METHODOLOGY

A. Data Collection

The initial step in any machine learning workflow is data collection, which forms the foundation for model training and evaluation. For this research, we used the publicly available Plant Village dataset, a comprehensive dataset designed for classification and detection of plant diseases.

B. Preprocessing

Normalization of image size and color intensity is a crucial pre-processing step that enhances model training efficiency. Standardizing image dimensions ensures uniform input size, simplifying batch processing, and model architecture. Normalizing pixel values scales the intensity range (e.g. 0 to 1), reducing computational complexity and aiding gradient-based optimization.

C. Model Architecture

1) *ResNet Model*: The ResNet model aims to solve the problem of vanishing gradient and train deep neural networks by revealing residual connections. Key components include:

- **Input Layer**: Standard convolution layer with a large kernel (e.g. 7x7) for initial feature extraction, followed by batch normalization and max-pooling.
- **Residual Blocks**:
 - **Convolutional Layers**: Each block consists of multiple convolutional layers with small kernels (e.g., 3x3).
 - **Skip Connections (Identity Mapping)**: Allow the input of a block to bypass one or more layers and be added directly to the block's output, facilitating gradient flow during backpropagation.
 - **Batch Normalization**: Normalizes activations to stabilize and accelerate training.
 - **ReLU Activation**: Nonlinearity is introduced using the ReLU function.

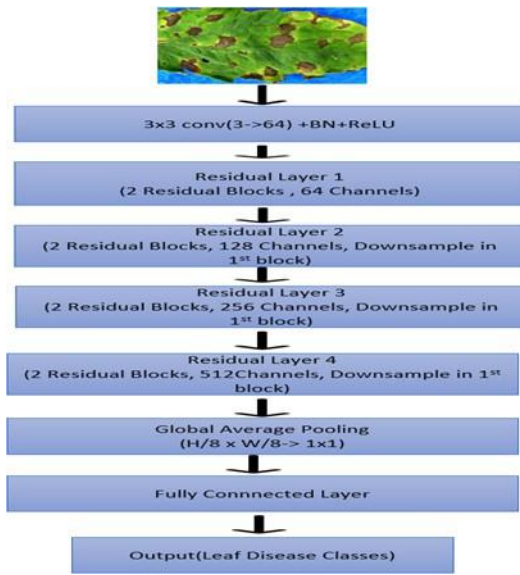


Fig. 1. ResNet Model Architecture used for Plant Disease Detection

- Bottleneck Layers (in deeper ResNets): A three-layer structure within residual blocks: Dimensionality reduction by 1x1 convolution. Feature extraction by 3x3 convolution. Dimensionality restoration by 1x1 convolution.
- Pooling Layers: Spatial downsampling using max-pooling or stride-based convolutions in certain residual blocks.
- Fully Connected Layers: Global average pooling followed by a fully connected layer using softmax activation.

2) MobileNet Model:

- Input Layer: The process begins with an image of a leaf. This image serves as input to the Convolutional Neural Network (CNN) for further analysis.
- Feature Extraction: Convolution and Pooling : The CNN uses multiple layers to identify important features from the image:
 - First Convolutional Layer: Applies a 3×3 convolution filter to extract simple patterns from the image. The input has 3 channels (RGB) and produces 32 feature maps as output. Max Pooling (2×2) reduces the size of the image, keeping only the most important information.
 - Second Convolutional Layer: Uses 64 filters to detect more detailed features. Another Max Pooling (2×2) layer further compresses the image.
 - Third Convolutional Layer: Expands to 128 filters to capture more complex patterns. Max Pooling (2×2) reduces dimensions again.
 - Fourth Convolutional Layer: Increases to 256 filters, refining the extracted features even further. The final Max Pooling (2×2) step reduces unnecessary details,

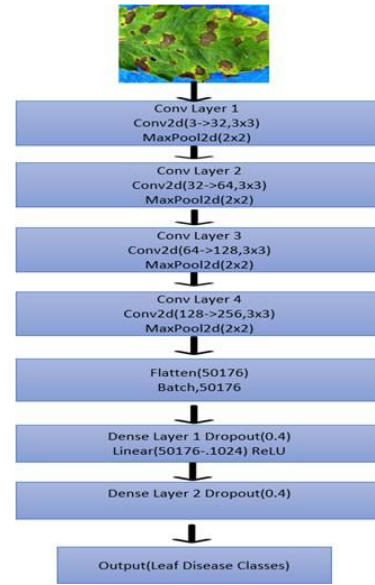


Fig. 2. MobileNet Model Architecture used for Plant Disease Detection

making the data more manageable.

- Transformation: Flattening and Normalization: The processed image is converted into a single long vector (size: 50176). Batch normalization is applied to keep the training process stable and efficient.
- Fully Connected Layers:
 - First Dense Layer: Uses Dropout (40%) to avoid overfitting. A fully connected layer (50176 → 1024 neurons) applies ReLU activation to introduce non-linearity.
 - Second Dense Layer: Another Dropout (40%) to improve generalization.
- Output: Classification of Leaf Disease
The final layer makes a prediction, categorizing the leaf based on its disease symptoms.

D. Training:

The model is trained using a dataset of plant leaf images, each labeled as healthy or diseased. During training, it identifies important visual patterns such as color changes, texture differences, and shape irregularities that indicate disease presence. The learning process occurs in multiple stages, where the model refines its ability to recognize features at different levels. Using back-propagation and optimization techniques, it continuously adjusts its internal parameters to reduce errors. With each training cycle, the model enhances its accuracy, enabling it to effectively classify leaf diseases.

IV. PRACTICAL IMPLEMENTATION OF THE PROPOSED PLATFORM

A. Model Used

The ResNet model was trained on a dataset of plant leaf images to accurately classify diseases. The trained model was integrated into a web-based system to make it accessible to users.

B. Web Integration

A Flask-based backend was developed to process images uploaded by users. The system analyzes each image and returns the following:

- The type of plant disease detected.
- Recommended actions to prevent further crop damage.

C. User Interface

The platform features a simple and easy-to-use interface where farmers can upload leaf images. The system processes the image and displays results in a clear, understandable format.

D. Challenges and Solutions

- **Computational Complexity:**
ResNet requires significant computational power, making deployment on low-resource devices challenging.

Solution: Using Lightweight Models like MobileNet: MobileNet is an efficient deep learning model optimized for mobile and embedded devices. It uses depthwise separable convolutions to reduce computation while maintaining good accuracy.

- **Scalability and Adaptability**
The dataset used for training may not fully represent all possible plant species and disease variations, affecting generalizability.

Solution: Continuous Data Collection: Integrate crowd-sourced image collection from farmers to expand dataset diversity.

V. EXPERIMENTAL ANALYSIS

A. Dataset

The dataset (PlantVillage) consists of more than 50,000 images of leaves of various plants, covering 14 crop species and more than 25 different plant diseases. These images are labeled as either healthy or diseased, with each disease category corresponding to a specific plant species. The resolution of the images in the dataset typically varies, but they are generally high-resolution, with most images ranging from 300x300 pixels to 256x256 pixels. This variation in resolution allows for training models that can handle different levels of detail, ensuring robustness in disease detection across various

plant species and conditions. The dataset contains both healthy and unhealthy images of Apple, Corn, Cherry, Blueberry, Grapes, Pepper Bell, Peach, Orange, Potato, Tomato, Soybean, Squash, Strawberry and Raspberry which have diseases like leaf blight, early blight, late blight, black rot, common rust, bacterial infection(s), cercospora leaf spot, haunglonbing, septoria leaf spot, spider mitea and powdery mildew.

B. Performance Analysis

A comparison of model performance is shown in Table I.

Model	Accuracy
ResNet	99.23%
MobileNet	96.97%
CNN	90.02%

TABLE I
PERFORMANCE COMPARISON OF DIFFERENT DEEP LEARNING MODELS FOR PLANT DISEASE DETECTION

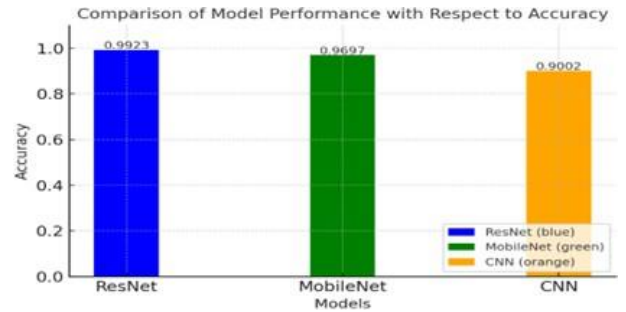


Fig. 3. Comparison of Model Performance wrt Accuracy

VI. CONCLUSION AND FUTURE SCOPE

A. Conclusion

Our study focuses to assist farmers in identifying and diagnosing diseases in plants using deep convolutional neural networks (CNN). Among the models evaluated (CNN, MobileNet, ResNet), ResNet was chosen for its exceptional accuracy and ability to classify diseases with precision. Trained on the Plant Village dataset, ResNet exhibited excellent performance, making it a reliable option for disease detection, even though it requires greater computational resources. Thanks to its deeper architecture and residual connections, ResNet can analyze complex patterns in images, making it effective at recognizing subtle disease indicators. While it consumes more memory and processing power, this trade-off is justified by its consistent accuracy and reliability. ResNet, offering superior accuracy, making them ideal for farmers in rural areas, enabling rapid identification of plant pest diseases and timely action to reap agricultural benefits. The research demonstrates the potential of deep learning to improve agriculture and in supporting permaculture.

B. Future Scope

These are the main areas where the research can be further developed and expanded:

- **Disease Detection using Hyperspectral Images:** The work can now be extended to identify more diseases using the ability of hyperspectral imaging to identify small changes in tissue which may not be visible on standard imaging. This leads to early disease detection and precise disease control strategies, useful for large-scale agricultural and plant pathology research.
- **Predicting disease severity:** The current model predicts the presence or absence of disease on leaves, but it is possible to extend this to quantify damage or infection using state-of-the-art machine learning to better understand the infection. This will allow for intervention plans, improved resource utilization and improved crop management.
- **Precision Agriculture:** Plant disease detection can be integrated into precision agriculture systems, allowing for targeted treatments and interventions. By combining disease detection with data on nutrient levels, soil moisture, and weather conditions, farmers can optimize resource usage and minimize the spread of diseases.
- **Real-time monitoring:** Developing real-time monitoring using Internet of Things (IoT) Devices and Remote Sensing Technology can aid in timely detection and intervention to control crop diseases and reduce crop losses.

C. End-User Experience and Feedback

While a formal field study is yet to be conducted, early feedback from test users indicates: The platform is easy to use, even for those with minimal technical knowledge. Farmers find the disease identification feature helpful for early diagnosis.

REFERENCES

- [1] Zhiyong Xiao et al., "Leaf Disease Detection Based on Lightweight Deep Residual Network and Attention Mechanism," 2023.
- [2] Divine Senanu Ametefe et al., "Enhancing leaf disease detection accuracy through synergistic integration of deep transfer learning and multimodal techniques," 2024.
- [3] Aisha Ahmed AlArfaj et al., "Plant Leaf Disease Detection and Classification Using Image Processing," International Journal of Research and Engineering, 2018.
- [4] Amrita S., Natasha Raul, and Tulshan. "Using machine learning to detect plant leaf disease." Tenth International Conference on Communication, Networking Technologies and Computing to be held in 2019 (ICCCNT). IEEE, 2019.
- [5] Deepalakshmi, P., K. Lavanya, and Parvathaneni Naga Srinivasu, using the CNN method, were able to detect plant leaf disease. IJISMD 12.1 (2021): 1–21. International Journal of Information System Modeling and Design.
- [6] Tian, Mengxiao, Hong Chen, and Qing Wang, using deep learning identified flowers, Journal of Physics: Conference Series, Vol. 1237, No. 2, IOP Publishing, 2019.
- [7] Thi Thanh Nhan et al., "Flower species identification using deep convolutional neural networks," 2016 Regional Conference on Computer and Information Engineering, sponsored by AUN/SEED-Net.
- [8] Emanuel Cortes, "Plant disease classification using convolutional networks and generative adversarial networks," (2017).
- [9] Y. N. Norasma, R. N. Athirah, and M. R. Ismail. "The development of an Android crop management application for smart farming," IOP Conference Series: Earth and Environmental Science, Vol.540, No.1, 2020.
- [10] Srdjan, Sladojevic and others. "Deep neural networks based recognition of plant diseases by leaf image classification." Computational intelligence and neuroscience 2016 (2016).
- [11] Mohammed Brahimi, "Deep Learning for Tomato Diseases: Classification and Symptoms Visualization"
- [12] Singh, & Singh, "Plant disease detection and classification techniques: a comparative study of the performances."
- [13] Jasim, H., & Al-Tuwaijari, "Plant Leaf Diseases Detection and Classification Using Image Processing and Deep Learning Techniques"
- [14] Arora, & Singh, "Detection of plant leaf diseases using image segmentation and soft computing techniques"
- [15] Wang, et al. "Plant diseases and pests detection based on deep learning"
- [16] Srdjan Sladojevic, Marko Arsenovic, "Deep Neural Networks Based Recognition of Plant Diseases by Leaf Image Classification"
- [17] Barbedo, J. G. A., "Plant disease identification from individual lesions and spots using deep learning"
- [18] Zhang, "Plant Disease Detection and Classification by Deep Learning"
- [19] Hassouni, "An automatic plant leaf disease identification using DenseNet-121 architecture with a mutation-based Henry Gas Solubility Optimization algorithm"
- [20] Usharani, "Automated Plant Disease Detection through Deep Learning for Enhanced Agricultural Productivity"